مسارات الـ EMV في EasyLink

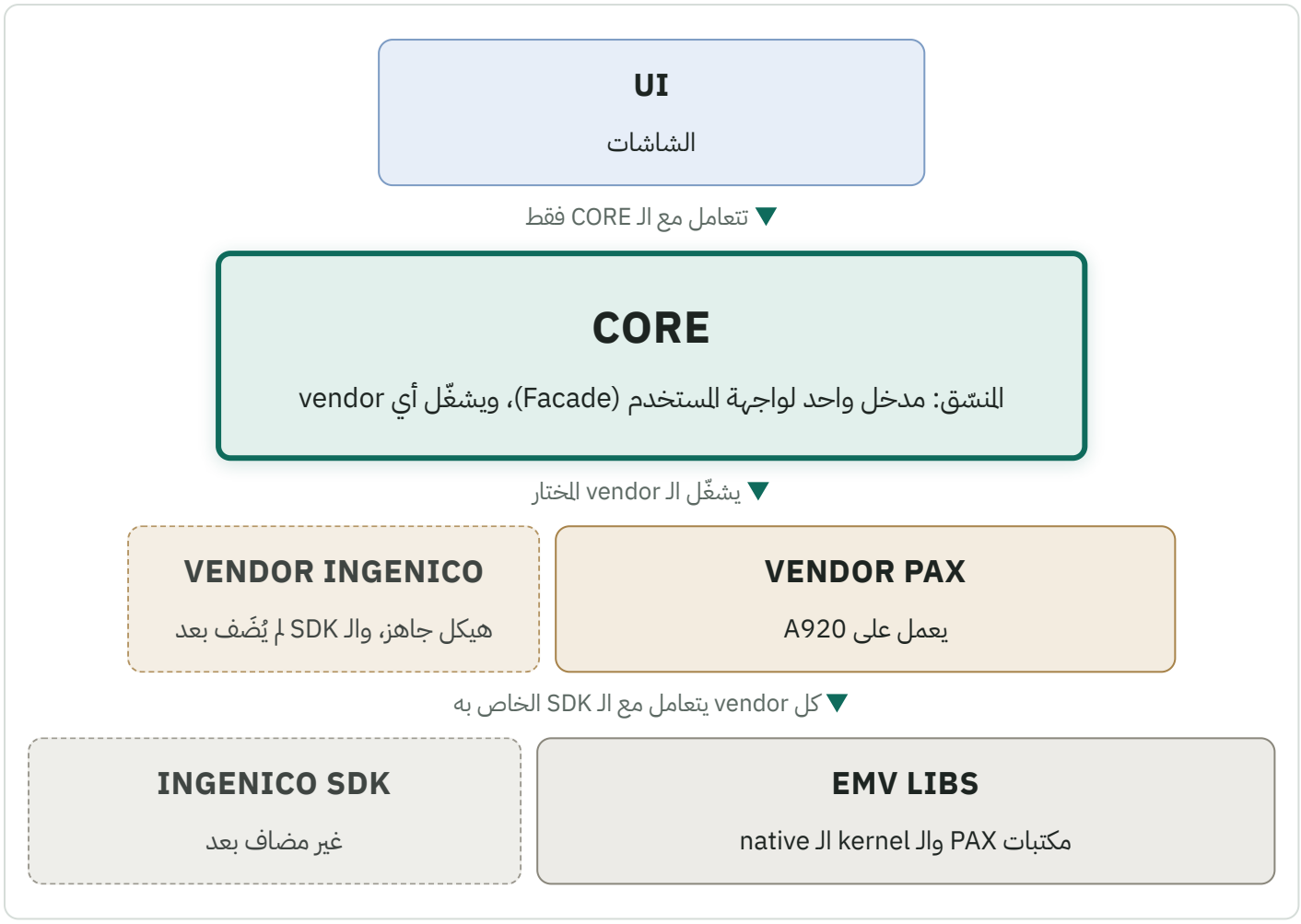
رحلة العملية من زر البيع حتى نهايتها: البطاقة، والـ kernel، والبنك، ثم التأكيد أو الإلغاء عند البنك. كل اسم في الصفحة موجود في الكود، وكل سهم استدعاء حقيقي. الشرح كله على فرع `transactionStepObserver`.

---

1. المكوّنات: من يملك ماذا
2. العملية من البداية إلى النهاية
3. خطوات الـ EMV: goToStep و onXxx
4. حين ينتظر الـ kernel شخصًا: askStep
5. الطريق إلى البنك
6. مسار الـ contact
7. النتيجة والنهاية الآمنة
8. مسار الـ contactless
9. الـ EMV params
10. مبادئ البرمجة في المشروع
11. استخدامنا كـ SDK

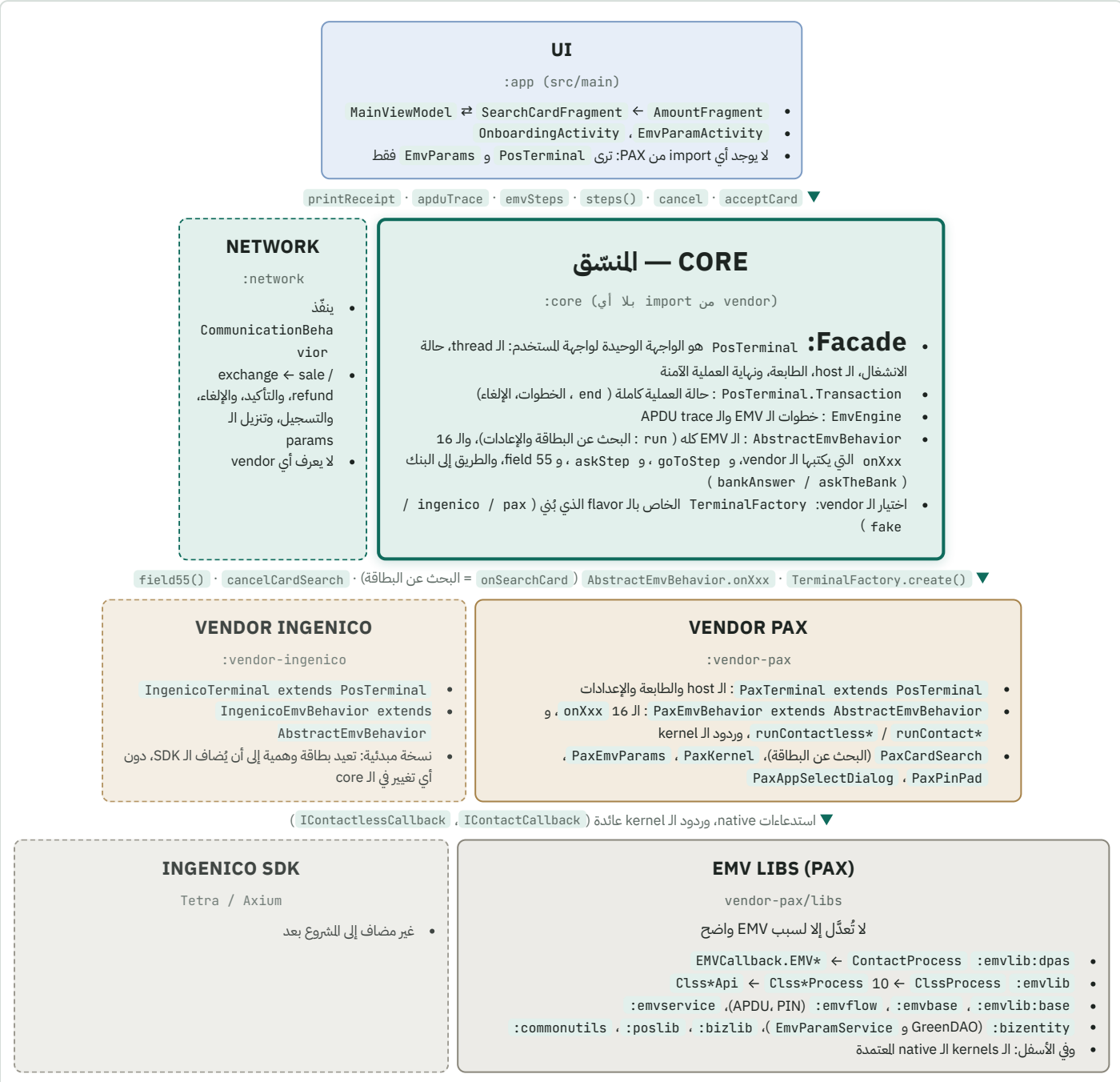
1. المكوّنات: من يملك ماذا

لكل مسؤولية مالك واحد. الشاشة لا تعرف PAX، والـ EMV لا يعرف البنك ولا الـ threads، و `PosTerminal` هو الباب الوحيد الذي تدخل منه الشاشة.


UI
الشاشات
▼ تتعامل مع الـ CORE فقط
CORE
المنسّق: مدخل واحد لواجهة المستخدم (Facade)، ويشغّل أي vendor
▼ يشغّل الـ vendor المختار
VENDOR INGENICO
هيكل جاهز، والـ SDK لم يُضَف بعد
VENDOR PAX
يعمل على A920
▼ كل vendor يتعامل مع الـ SDK الخاص به
INGENICO SDK
غير مضاف بعد
EMV LIBS
مكتبات PAX والـ kernel الـ native

الطبقات بالتفصيل

كل مربع هو module في الكود، وتحته أهم الكلاسات. ما بين الطبقتين هو الواجهة الوحيدة التي تمر من طبقة إلى التي تحتها.



- **الاستدعاءات تتجه إلى الأسفل فقط.** ما يصعد إلى الأعلى هو الخطوات (`Transaction.steps()` ، `emvSteps`)، وردود الـ kernel التي تعود إلى الـ vendor الخاص بها.
- **إضافة vendor جديد** تعني module جديدًا فيه `TerminalFactory` ، وكلاس يرث `PosTerminal` (الإعدادات)، وكلاس يرث `AbstractEmvBehavior` (الخطوات، ومنها البحث عن البطاقة). لا تتغير واجهة المستخدم ولا الـ CORE (`doc/VENDOR.md`).
- يوجد أيضًا `:vendor-fake` للاختبار، ويثبت أن الـ CORE يعمل دون أي SDK.

مكوّنات كل طبقة

كل بطاقة تمثل طبقة من الهرم، وبداخلها الـ packages، وكل كلاس ووظيفته في سطر.

UI `:app (src/main)`

الشاشات. تطلب من الـ CORE وتعرض ما يعود منه، ولا تحتفظ بأي حالة للعملية ولا تعرف أي vendor.

بدء التطبيق

`EmvEnhanceApp`

ينشئ `PosTerminal` واحدًا طوال عمر التطبيق (`TerminalFactory.create`)، ويحدد عنوان الـ host

عملية البيع

`MainActivity`

الشاشة الرئيسية: أزرار فقط

`AmountActivity` / `AmountFragment`

نوع العملية والمبلغ، ثم شاشة البطاقة

`SearchCardFragment`

شاشة العملية: الشريط العلوي، الإلغاء، والنتيجة

`MainViewModel`

يربط `PosTerminal` بالـ LiveData: `acceptCard` ، `cancel` ، الخطوات، الـ trace

`Event`

قيمة تُقرأ مرة واحدة (بسبب تدوير الشاشة)

الإعدادات

`EmvParamActivity` / `EmvParamListFragment`

الـ EMV params: من الـ cache أو من قاعدة البيانات، والتنزيل

`OnboardingActivity`

تسجيل الجهاز عند الـ host، و mToken، و PIN bypass

`CountryListActivity`

الدولة والعملة

CORE — المنسّق `:core`

العملية من بدايتها إلى نهايتها، بالكود نفسه لأي vendor: واجهة المستخدم تتعامل معه وحده، وهو يتعامل مع الـ vendor من خلال واجهات (abstract classes و interfaces).

`terminal` — الجهاز

`PosTerminal`

الـ Facade: المدخل الوحيد لواجهة المستخدم. thread واحد، حالة الانشغال، الذهاب إلى البنك (`sendToBank`)، الطابعة

`PosTerminal.Transaction`

عملية واحدة: حالتها (`end`)، وخطواتها (يشغّلها `TransactionStepObserver`)، والإلغاء، والطلبات المرسلة إلى البنك ونهايتها الآمنة

`EmvBehavior`

الواجهة: `run` · `cancelCardSearch` · `dispose`

`BankRequest`

طريق الـ EMV إلى البنك: `send(config)` ؛ يعطيه `PosTerminal` للـ behavior بـ `attach(this::sendToBank)`

`AbstractEmvBehavior`

الـ EMV المشترك: `run` (المحاولات والبحث والإعادة: `requestRetry` / `retryCount()`، `goToStep` ، `askStep` ، 55 field، والـ 16 `onXxx` التي يكتبها الـ vendor. موجود هنا لأنه يستدعي `Transaction` باستدعاءات package-private

`TerminalBusyException`

محاولة بدء عملية بينما أخرى قيد التنفيذ

`transaction` — العملية

`TransactionConfig`

النوع، المبلغ، طريقة إدخال البطاقة، field 55، الـ PIN block، `asyncRequestId`

`TransactionType`

sale / refund / cash in

`TransactionStep / Event`

مراحل العملية: البدء، انتظار البطاقة، الاتصال بالـ host، الموافقة، التأكيد أو الإلغاء... والخطوات التي تقود العملية (البدء، الإلغاء، النتيجة، التأكيد / الإلغاء عند الـ host، النهاية) لكل منها دالة

`emv` — الـ EMV

`EmvEngine`

يرسل كل خطوة EMV إلى من يشغّلها وإلى الشريط العلوي، ويحتفظ بالـ APDU trace. لا علاقة له بحالة العملية ولا بإعادة المحاولة ولا بالبنك

`EmvStep / Event`

الخطوات الـ 17 الخاصة بالـ kernel (EMV Book 3)

`EmvTransactionResult`

ما قُرئ من البطاقة ويحتاجه الـ host

`Field55`

يبني 55 field من الـ tags بالطريقة نفسها لكل vendor

`ApduTrace`

سجل الـ APDU والخطوات (في نسخة الـ debug فقط)

`card` — البطاقة

`CardPresence / CardSearchListener`

البطاقة التي عُثر عليها (`isEmv()` / `isNonEmv()`)، وأحداث البحث

`EntryMethod`

chip / tap / swipe / manual

`host` — الواجهات الخارجية

`CommunicationBehavior`

الـ host: `connectToHost` · `reverse` · `acknowledge`

`AuthResult`

رد الـ host: approved / declined / failed / notSent

`PrinterBehavior`

الطابعة

`HostDefaults`

host وطابعة وهميان للعرض التجريبي

`cardholder` — انتظار حامل البطاقة

`AbstractAppSelectDialog`

اختيار التطبيق على البطاقة: الانتظار والمهلة، ولا يختار الأول تلقائيًا أبدًا

`PinPadBehavior / AbstractPinPadBehavior / PinKey`

شاشة الـ PIN: النقاط والأزرار فقط، والتشفير من مهام الـ SDK

`util · param`

`EmvParams`

واجهة شاشة الـ EMV params

`EmvLog`

السجلات

NETWORK `:network`

التواصل مع الـ host الخاص بنا. ينفّذ `CommunicationBehavior` ولا يعرف أي vendor.

عملية البيع

`SaleCommunicationBehavior`

sale / refund ← exchange، والتأكيد والإلغاء، ويحدد لكل فشل: هل يمكن أن يكون الطلب قد وصل إلى الـ host أم لا

`HostApiConnection` / `HostApiClient`

الـ endpoints (Retrofit) والـ client

`HostHeaders` / `HostSettings` / `HostAppKeys`

الـ headers: الـ token، و mToken (يُعدَّل من الجهاز)، والمفاتيح الثابتة

`PosEntryModeCodes`

051 / 071 / 901 / 011

`tlv` · `crypto`

`RsaPublicKeyEncryptor`

يشفّر مفتاح العملية (TEK) بمفتاح الـ host

`AesEnvelopeCrypto`

يشفّر الطلب والرد ويفك تشفيرهما

`BerTlv` / `IssuerResponseFields`

يقرأ الـ ARPC والـ scripts من رد الـ host

`onboarding` · **params**

`OnboardingClient` / `OnboardingState`

تسجيل الجهاز، والمفاتيح، والـ token وتجديده

`EmvParamDownloadClient`

ينزّل ملف الـ params (البيانات فقط، والقراءة لدى PAX)

`EnvironmentProvider`

عنوان الـ host الخاص بالـ build

`PinBypassSettings`

هل الـ PIN bypass مسموح أم لا

VENDOR PAX `:vendor-pax`

PAX A920: ينفّذ واجهات الـ CORE باستخدام SDK الخاص بـ PAX. هذا هو المكان الوحيد الذي يظهر فيه اسم PAX.

البدء

`TerminalFactory`

المكان الوحيد الذي يُنشئ `PaxTerminal`

`VendorApplication`

الـ Application الخاص بـ PAX (الـ SDK يحتاج `BaseApplication`)

`PaxHardwarePermissions`

صلاحيات قارئات ICC / Mag / PICC

`PaxSetup`

الدولة والعملة، والبيانات الأولية عند أول تشغيل (خرجت من `PaxTerminal`)

البطاقة والـ EMV

`PaxTerminal`

`extends PosTerminal` : الـ host والطابعة الفعليان، والإعدادات

`PaxCardSearch`

البحث عن البطاقة (polling): عمل الخطوة `SEARCH_CARD` (`onSearchCard`)، و `stop()` عند الإلغاء

`PaxEmvBehavior`

`extends AbstractEmvBehavior`: الـ 16 `onXxx` (استدعاء native لكل خطوة)، والرد على أسئلة الـ kernel، و `field55()`

`PaxKernelSession`

ما عُرف خلال المحاولة الحالية، ويُنشأ من جديد مع كل محاولة

`PaxKernel`

كائنات الـ kernel: params، contact، contactless، mag

مساعدات الـ EMV

`PaxPinEntry`

الـ PIN: اللوحة، مفتاح الـ online PIN (AES) ولفّه بـ RSA للـ host، وقواعد الـ bypass. مفتاح الـ host العام يأتيه من `PaxTerminal` ، فلا يقرأ الـ EMV حالة الـ host بنفسه

`PaxHostAnswer`

رد الـ host بالشكل الذي يأخذه الـ kernel للـ 2nd GENERATE AC (`toOnlineResult`)

`PaxReaders`

إغلاق القارئات (mag، ICC، PICC) كلٌّ على حدة

`PaxCardData`

بيانات البطاقة كنص: الـ PAN من track 2 أو tag 5A، وإخفاء الـ PAN، والـ tags النصية (دوال ثابتة، وعليها أول اختبارات PAX)

الشاشات والأجهزة

`PaxPinPad`

شاشة الـ PIN

`PaxAppSelectDialog`

اختيار التطبيق

`PaxPrinter`

الطابعة الحرارية

`PaxEmvParams`

بيانات شاشة EMV Params (الـ cache / قاعدة البيانات / التنزيل)

VENDOR INGENICO `:vendor-ingenico`

الواجهات نفسها تمامًا، لكن بنسخة مبدئية: تعيد بطاقة ونتيجة وهميتين إلى أن يُضاف الـ SDK. عند إضافته لن يتغير الـ CORE ولا واجهة المستخدم.

`TerminalFactory` / `VendorApplication`
البدء

`IngenicoTerminal`
`extends PosTerminal` : الـ host والطابعة الوهميان

`IngenicoEmvBehavior`
`extends AbstractEmvBehavior`: بحث وهمي (`onSearchCard` ؛ لاحقًا Tetra / Axium)، وكل خطوة تنتقل إلى التالية

EMV LIBS (PAX)

`vendor-pax/libs/sdk — emvlib · emvbase · emvservice · bizlib · poslib · commonutils · vendor-pax/libs/ours — emvflow · bizentity`

الـ SDK الخاص بـ PAX والـ kernels الـ native المعتمدة، وكلها داخل الـ vendor في `vendor-pax/libs/` : المجلد `sdk/` هو كود PAX نفسه، و `ours/` هو كودنا (`emvflow` و `bizentity`). لا يتعامل معها إلا vendor-pax (مسارات Gradle مثل `:vendor-pax:libs:sdk:poslib` ، وتُكتب هنا مختصرة `:poslib`)، ولا تُعدَّل إلا لسبب EMV واضح.

الـ kernels

`:emvlib:dpas — ContactProcess`
الـ contact kernel: من `EMVAppSelect` إلى `EMVCompleteTrans` ، و `EmvCallBackListener`

`:emvlib — ClssProcess + Clss*Process`
الـ contactless kernel: لكل scheme (PayPass، PayWave، ...)

`:emvlib:base` / `:emvbase`
الكلاسات الأساسية، والـ params، والنتائج، والردود

الجهاز

`:emvflow`
EmvFlowRuntime (DAL)، EmvDeviceImpl (APDU، PIN، trace)

`:emvservice`
PinService، MagCardService

`:poslib` / `:commonutils` / `:bizlib`
أدوات PAX الخاصة بالجهاز

الـ params

`:bizentity`
GreenDAO (قاعدة البيانات)، `EmvParamService` (الـ cache)، قارئات الـ XML، `EmvParamUpdater` ، `ConfigInit` (أول تشغيل)

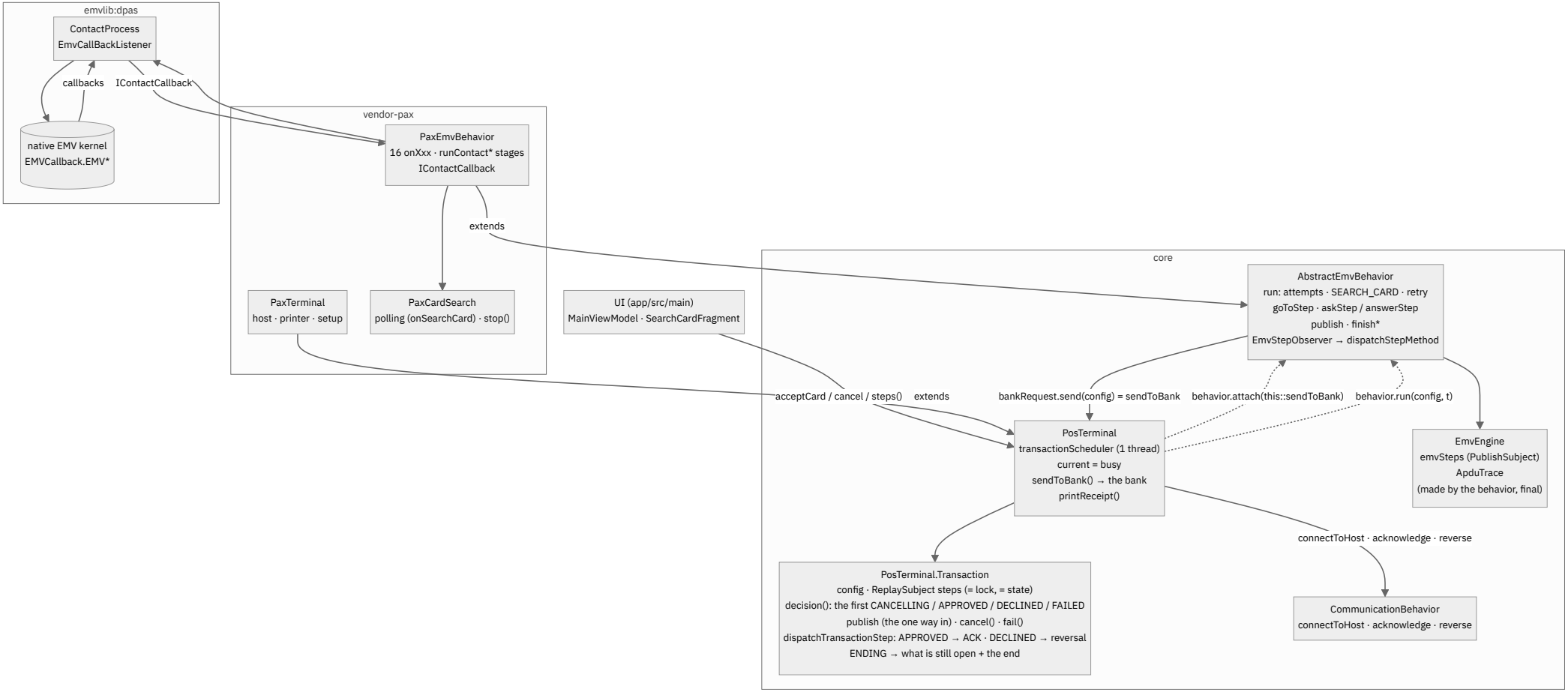
`PosTerminal` هو الجهاز نفسه

```
// PosTerminal = the POS machine
protected final AbstractEmvBehavior   behavior;       // the card: EMV (search + reader
protected final CommunicationBehavior communication;  // HTTP to the bank
protected final PrinterBehavior       printer;        // the receipt
private   final Scheduler             transactionScheduler; // one sale at a time, one t
```

الكلاس	مسؤول عن	ممنوع عليه
`PosTerminal`	عملية واحدة في كل مرة (`current` = مشغول)، والذهاب إلى البنك (`sendToBank`)، والطابعة	أي منطق EMV
`PosTerminal.Transaction`	حالة العملية كلها (`end`)، وخطواتها (`steps`)، والإلغاء، والطلبات المرسلة إلى البنك ونهايتها الآمنة	نشر أي شيء بعد انتهائها
`AbstractEmvBehavior`	الـ EMV المشترك: المحاولات، و `goToStep` ، و `askSte` p، و `askTheBank` / `bankAnswer` ، والـ 16 `onXx` x التي يكتبها الـ vendor	الـ threads، والانشغال، والبنك مباشرة
`PaxEmvBehavior`	الـ `onXxx` على PAX: استدعاء native لكل خطوة، وردود الـ kernel	أن يستدعي أحدٌ `onXxx` مباشرة
`EmvEngine`	شريط خطوات الـ EMV (`emvSteps`) والـ APDU trace	أي حالة للعملية
`SaleCommunicationBehavior`	الاتصال بالبنك: `connectToHost` (← token ← sale / refund exchange)، والتأكيد، والإلغاء؛ ويقرر هل وصل الطلب أم لا	أن يقرر التأكيد أو الإلغاء

من يتعامل مع من (الكلاسات)

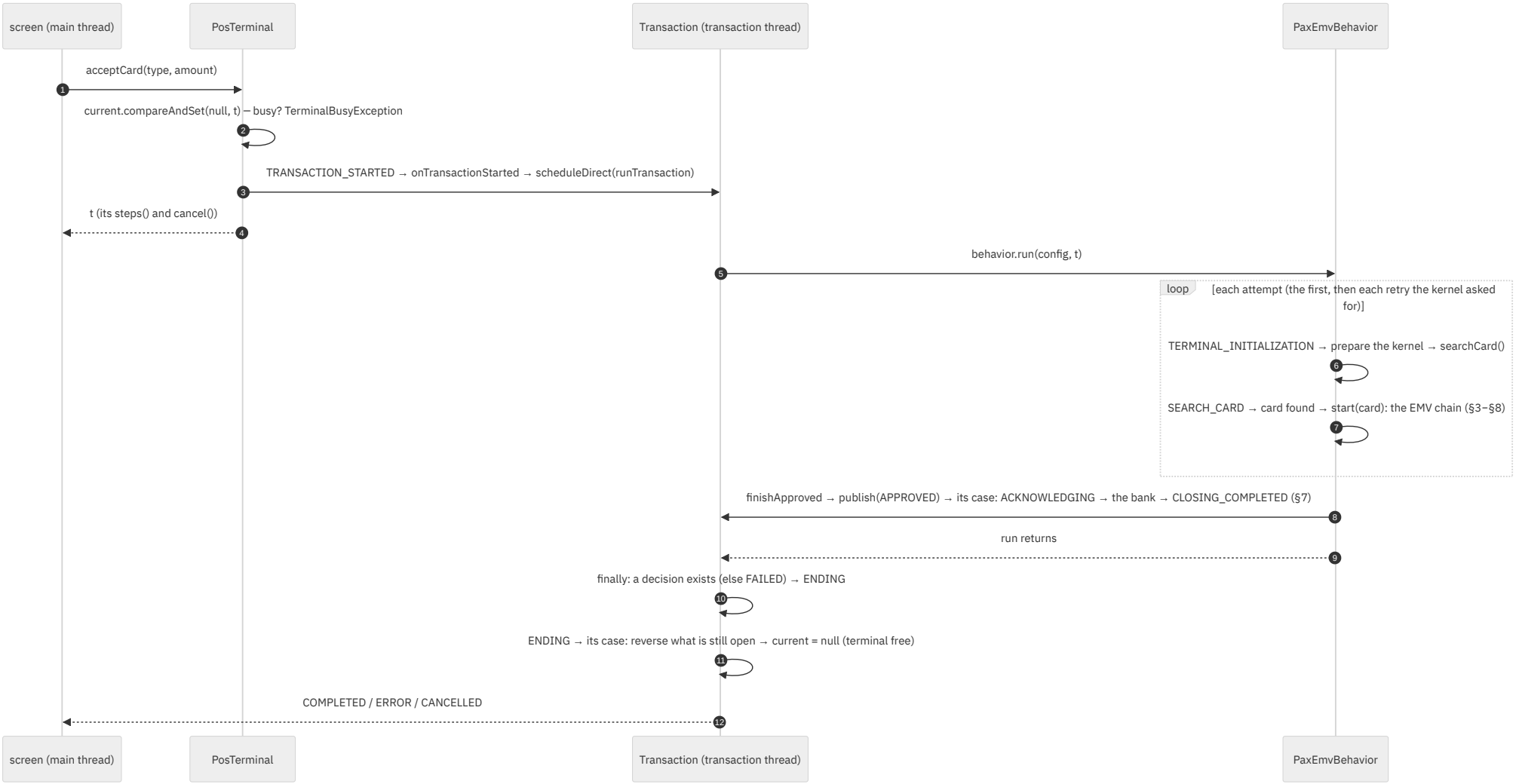
كل سهم استدعاء حقيقي. الأسهم المتقطعة تُوصَل مرة واحدة عند البداية.


emvlib:dpas
ContactProcess
EmvCallBackListener
callbacks
IContactCallback
native EMV kernel
EMVCallback.EMV*
vendor-pax
PaxEmvBehavior
16 onXxx · runContact* stages
IContactCallback
extends
PaxTerminal
host · printer · setup
PaxCardSearch
polling (onSearchCard) · stop()
UI (app/src/main)
MainViewModel · SearchCardFragment
core
AbstractEmvBehavior
run: attempts · SEARCH_CARD · retry
goToStep · askStep / answerStep
publish · finish*
EmvStepObserver → dispatchStepMethod
acceptCard / cancel / steps()
extends
bankRequest.send(config) = sendToBank
behavior.attach(this::sendToBank)
behavior.run(config, t)
PosTerminal
transactionScheduler (1 thread)
current = busy
sendToBank() → the bank
printReceipt()
EmvEngine
emvSteps (PublishSubject)
ApduTrace
(made by the behavior, final)
connectToHost · acknowledge · reverse
PosTerminal.Transaction
config · ReplaySubject steps (= lock, = state)
decision(): the first CANCELLING / APPROVED / DECLINED / FAILED
publish (the one way in) · cancel() · fail()
dispatchTransactionStep: APPROVED → ACK · DECLINED → reversal
ENDING → what is still open + the end
CommunicationBehavior
connectToHost · acknowledge · reverse

2. العملية من البداية إلى النهاية

العملية تسير **بخطواتها** (TransactionSteps). كل خطوة تدخل من باب واحد، `publish`، فتُحفظ في `Transaction.steps`، ثم يشغّل `TransactionStepObserver` الـ case الخاص بها في `dispatchTransactionStep`. ما ليس له case تقرير للشاشة فقط.

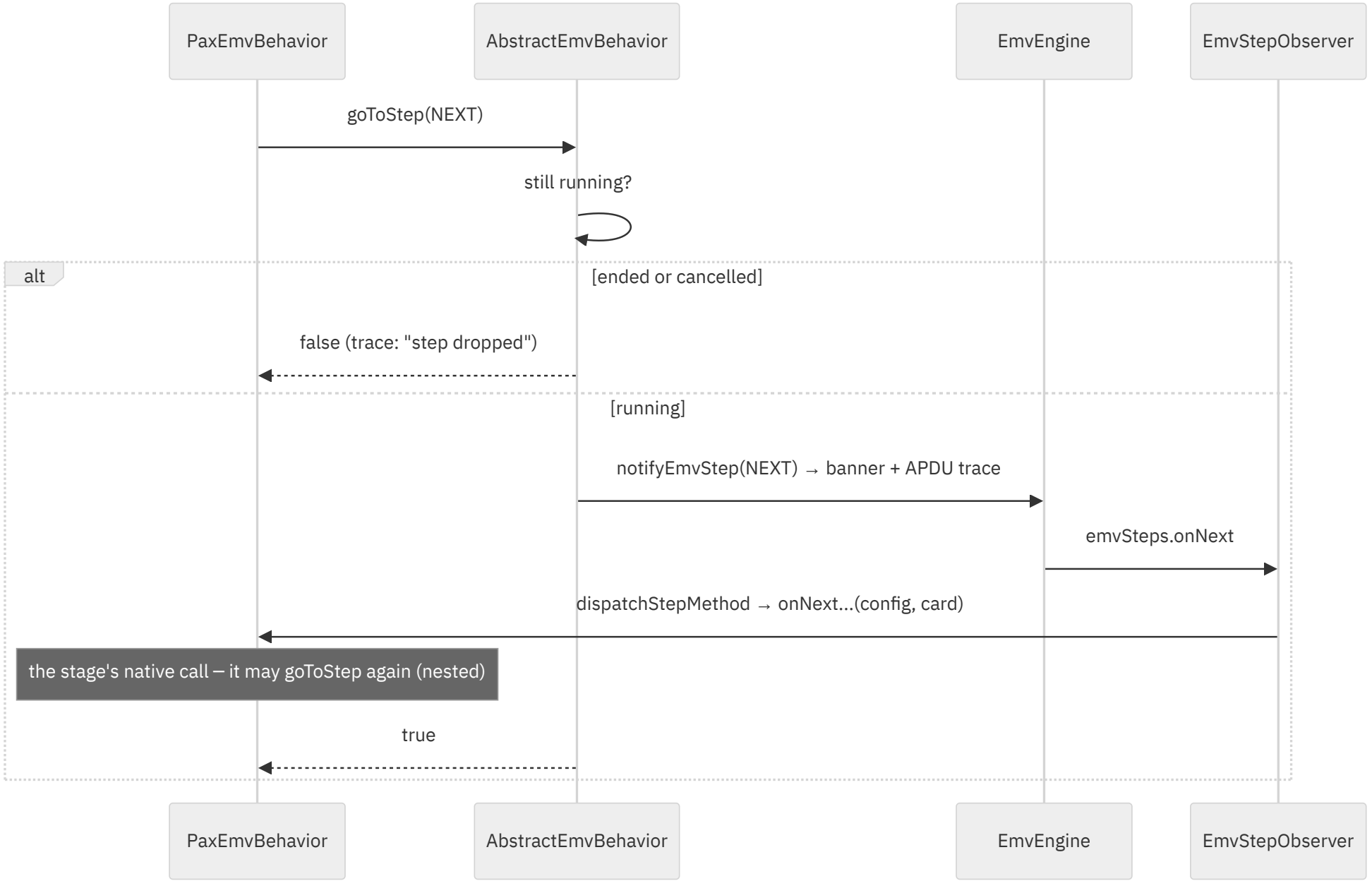
```
// TransactionStepObserver.onNext → dispatchTransactionStep(event)
switch (event.getStep()) {
    case TRANSACTION_STARTED: onTransactionStarted(event);       // → the transaction th
    case CANCELLING:          onCancelling();                    // stop the card search
    case APPROVED:            closeLast(ACKNOWLEDGING);          // the host + the card
    case DECLINED:            closeLast(REVERSING);              // the host approved, t
    case ENDING:              onEnding();                        // reverse what is stil
    default:                  // a report: ONLINE_*, CARD_*, FAILED, ACKNOWLEDGING / REV
                              // CLOSING_COMPLETED, COMPLETED / ERROR / CANCELLED…
}
```


screen (main thread)
PosTerminal
Transaction (transaction thread)
PaxEmvBehavior
acceptCard(type, amount)
1
current.compareAndSet(null, t) — busy? TerminalBusyException
2
TRANSACTION_STARTED → onTransactionStarted → scheduleDirect(runTransaction)
3
t (its steps() and cancel())
4
behavior.run(config, t)
5
loop
[each attempt (the first, then each retry the kernel asked for)]
TERMINAL_INITIALIZATION → prepare the kernel → searchCard()
6
SEARCH_CARD → card found → start(card): the EMV chain (§3–§8)
7
finishApproved → publish(APPROVED) → its case: ACKNOWLEDGING → the bank → CLOSING_COMPLETED (§7)
8
run returns
9
finally: a decision exists (else FAILED) → ENDING
10
ENDING → its case: reverse what is still open → current = null (terminal free)
11
COMPLETED / ERROR / CANCELLED
12
screen (main thread)
PosTerminal
Transaction (transaction thread)
PaxEmvBehavior

- **الانشغال:** يُفحص على الـ main thread نفسه، فتُرفض عملية ثانية فورًا بـ `TerminalBusyException`، بلا طابور.
- **حالة العملية هي خطواتها:** لا field. أول خطوة "قرار" تُسلَّم هي الحالة (`decision()`): `CANCELLING` أو `APPROVED` أو `DECLINED` أو `FAILED`. لا قرار = العملية شغالة. أول قرار يكسب وما بعده يُتجاهل. النهاية تُسلَّم أخيرًا من القرار: `CANCELLED` / `COMPLETED` / `ERROR` (بالرسالة نفسها). `FAILED` خطوة جديدة لأن `ERROR` يجب أن تبقى آخر خطوة.
- **الإلغاء** (`Transaction.cancel()`): تُنشر `CANCELLING` (القرار)، فتتوقف قراءة البطاقة، ويتوقف الـ EMV عند أول `goToStep`. لا يُقاطَع الـ thread أبدًا.
- **إعادة المحاولة** (fallback، tap again، See Phone): يطلبها الـ EMV بـ `requestRetry(...)` ثم يرجع الـ kernel، فيبدأ `run` محاولة جديدة من `TERMINAL_INITIALIZATION`، في العملية نفسها.
- **النتيجة ليست النهاية:** `APPROVED` / `DECLINED` / `FAILED` تُنشر فور معرفتها، أما `COMPLETED` فبعد التأكيد / الإلغاء عند البنك وتحرير الجهاز.
- ترى الشاشة مثلًا: `ONLINE_REQUIRED → ONLINE_PROCESSING → ONLINE_COMPLETED → APPROVED → ACKNOWLEDGING → CLOSING_COMPLETED → ENDING → COMPLETED`.
- ⚠ الـ cases تعمل والـ lock مأخوذ، ومعها نداءات البنك (التأكيد / الإلغاء). لذلك `publish` تنظر في القرار قبل أخذ الـ lock، فلا ينتظر `cancel` الإغلاقات. والخطوات التي قد تنتظر البنك أو شخصًا (`ONLINE_PROCESSING`، و Single Tap مستقبلًا) تبقى تقارير لا دوال.

3. خطوات الـ EMV: كيف يشغّل `goToStep` الدالة `onXxx`

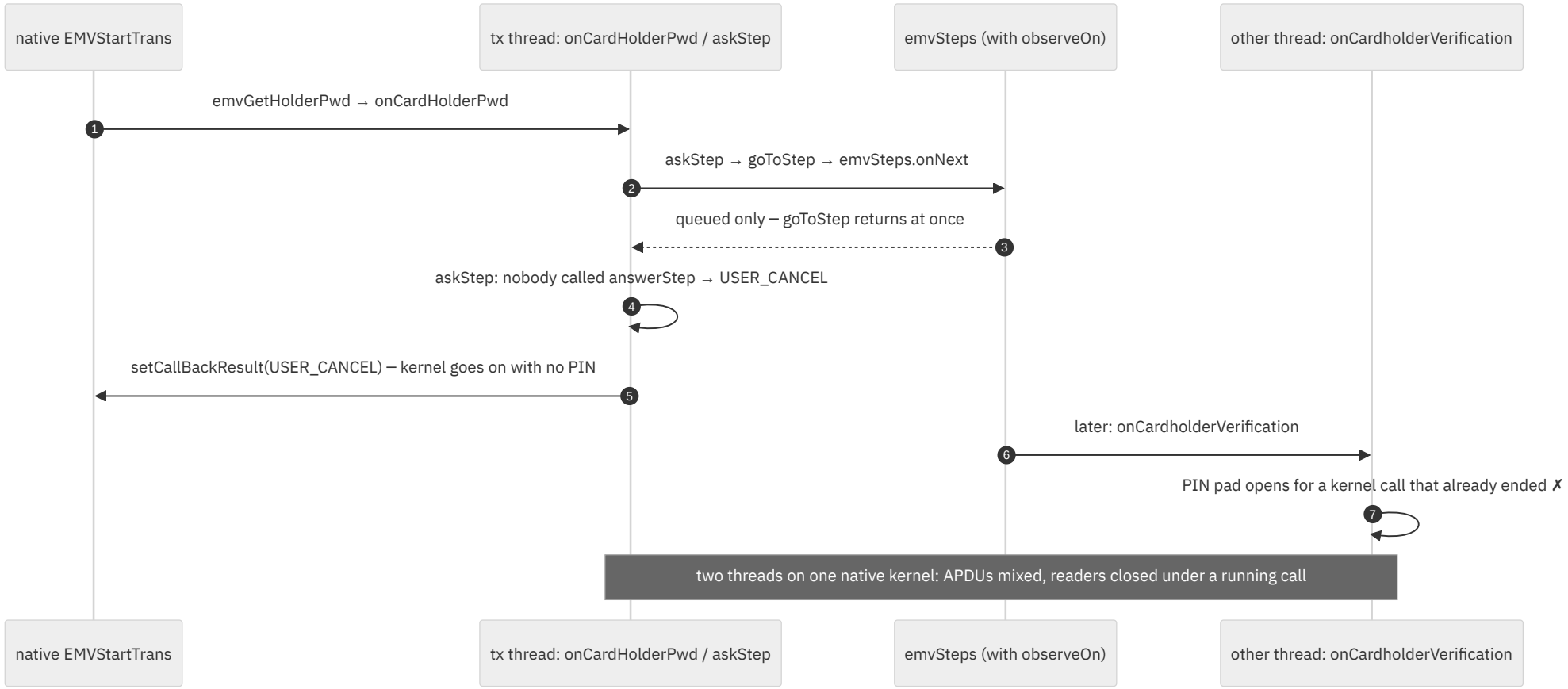
لا يستدعي أحد `onXxx` مباشرة. `goToStep(STEP)` تنشر الخطوة على `emvSteps`، فيشغّل `EmvStepObserver` دالتها على الـ thread نفسه، وتنتهي قبل أن تعود `goToStep`. إن كانت العملية قد انتهت أو أُلغيت تُسقط الخطوة وتعيد `goToStep` القيمة `false`.


PaxEmvBehavior
AbstractEmvBehavior
EmvEngine
EmvStepObserver
goToStep(NEXT)
still running?
alt
[ended or cancelled]
false (trace: "step dropped")
[running]
notifyEmvStep(NEXT) → banner + APDU trace
emvSteps.onNext
dispatchStepMethod → onNext...(config, card)
the stage's native call — it may goToStep again (nested)
true
PaxEmvBehavior
AbstractEmvBehavior
EmvEngine
EmvStepObserver

- **البداية:** `TERMINAL_INITIALIZATION` تجهّز الـ kernel ثم تنادي `searchCard()` بنفسها، فتبدأ `SEARCH_CARD`. إن فشل التجهيز تنتهي العملية بـ ERROR، ولا بحث.
- **يجب أن يبقى التشغيل متزامنًا** (بلا `observeOn` وبلا serialize): لو تأخرت الدالة إلى thread آخر لعادت `goToStep` قبل أن تعمل، فيرد `askStep` بـ "إلغاء" قبل أن تُفتح شاشة الـ PIN، وتتداخل استدعاءات native على kernel واحد.
- الشاشة وحدها تستخدم `observeOn(main)` على `emvSteps`، لأنها تعرض الشريط فقط ولا ينتظرها أحد.

لماذا يجب أن يبقى التشغيل متزامنًا (بلا `observeOn` وبلا serialize)

تعني `observeOn(X)` أن يوضع `onNext` في طابور ويُنفَّذ لاحقًا على thread آخر، وأن تعود `goToStep` فورًا. لو طُبّق ذلك على `EmvStepObserver` لعملت `onXxx` خارج الـ transaction thread، وهذا يكسر ثلاثة أشياء:

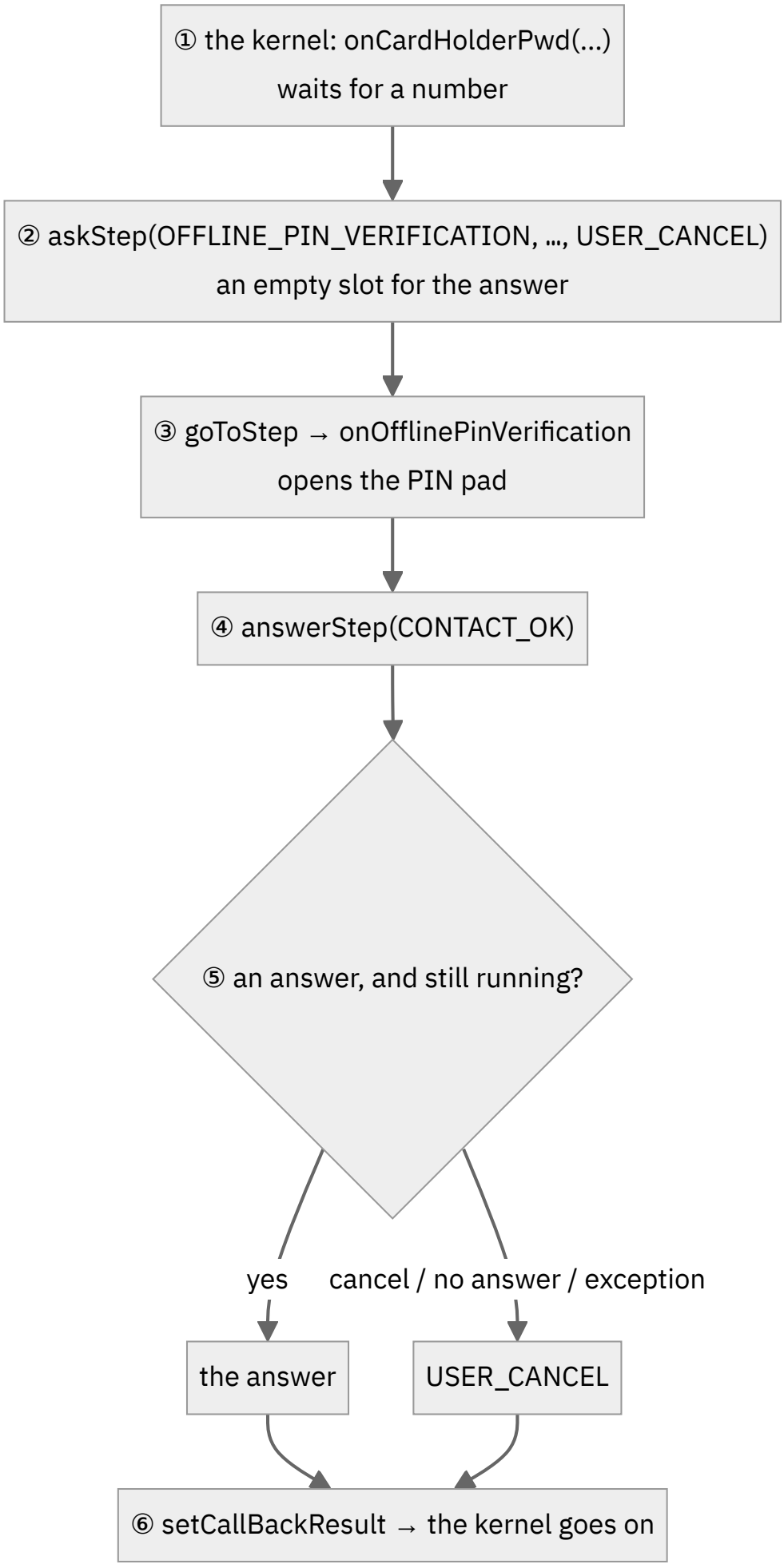

native EMVStartTrans
tx thread: onCardHolderPwd / askStep
emvSteps (with observeOn)
other thread: onCardholderVerification
emvGetHolderPwd → onCardHolderPwd
1
askStep → goToStep → emvSteps.onNext
2
queued only — goToStep returns at once
3
askStep: nobody called answerStep → USER_CANCEL
4
setCallBackResult(USER_CANCEL) — kernel goes on with no PIN
5
later: onCardholderVerification
6
PIN pad opens for a kernel call that already ended ✗
7
two threads on one native kernel: APDUs mixed, readers closed under a running call
native EMVStartTrans
tx thread: onCardHolderPwd / askStep
emvSteps (with observeOn)
other thread: onCardholderVerification

لو استخدمنا	ما الذي سيحدث	السبب
`observeOn` على الـ observer	تعيد `askStep` القيمة `USER_CANCEL` قبل أن تُفتح شاشة الـ PIN أو نافذة الاختيار	تعمل الخطوة بعد عودة `goToStep`، فلا يكون أحد قد أجاب في تلك اللحظة
`observeOn` على الـ observer	تداخل على الـ kernel: تستدعي `onXxx` دالة native (`EMVReadAppData`، `EMVCardAuth`) بينما الـ transaction thread ما زال داخل استدعاء native آخر، أو يغلق القارئات في `finishContactStage`	الـ kernel الخاص بـ PAX والقارئ ليسا thread-safe: يجب أن يجري كل شيء على thread واحد وبالترتيب
`observeOn` على الـ observer	يختل الترتيب: يكمل `runContactAppSelect` على افتراض أن مرحلة القراءة انتهت ونتيجتها في `tx.transResult`، وهي لم تنتهِ بعد	كل مرحلة تقرأ نتيجة سابقتها بعد عودة `goToStep`
`emvSteps.toSerialized()`	لا تعمل الخطوة التالية داخل الحالية، بل توضع في طابور حتى ينتهي `onNext` الحالي	كل `onXxx` تستدعي `goToStep(NEXT)` من داخل `onNext`، والـ serialize يؤجّل الاستدعاء المتداخل، فتعيد `askStep` قيمة الأمان وتجد المرحلة النتيجة ناقصة

بدونهما يصبح التشغيل مثل استدعاء دالة عادي: `onXxx` ← `onNext` ← `goToStep`، كله على الـ transaction thread، وينتهي قبل أن تعود `goToStep`. **لا مانع أن تستخدم واجهة المستخدم `observeOn(main)`** (`MainViewModel` على `emvSteps`)، لأنها تعرض الشريط العلوي فقط، ولا تشغّل EMV، ولا ينتظر أحد ردًا منها. هذا الشرط خاص بـ `EmvStepObserver` وحده.

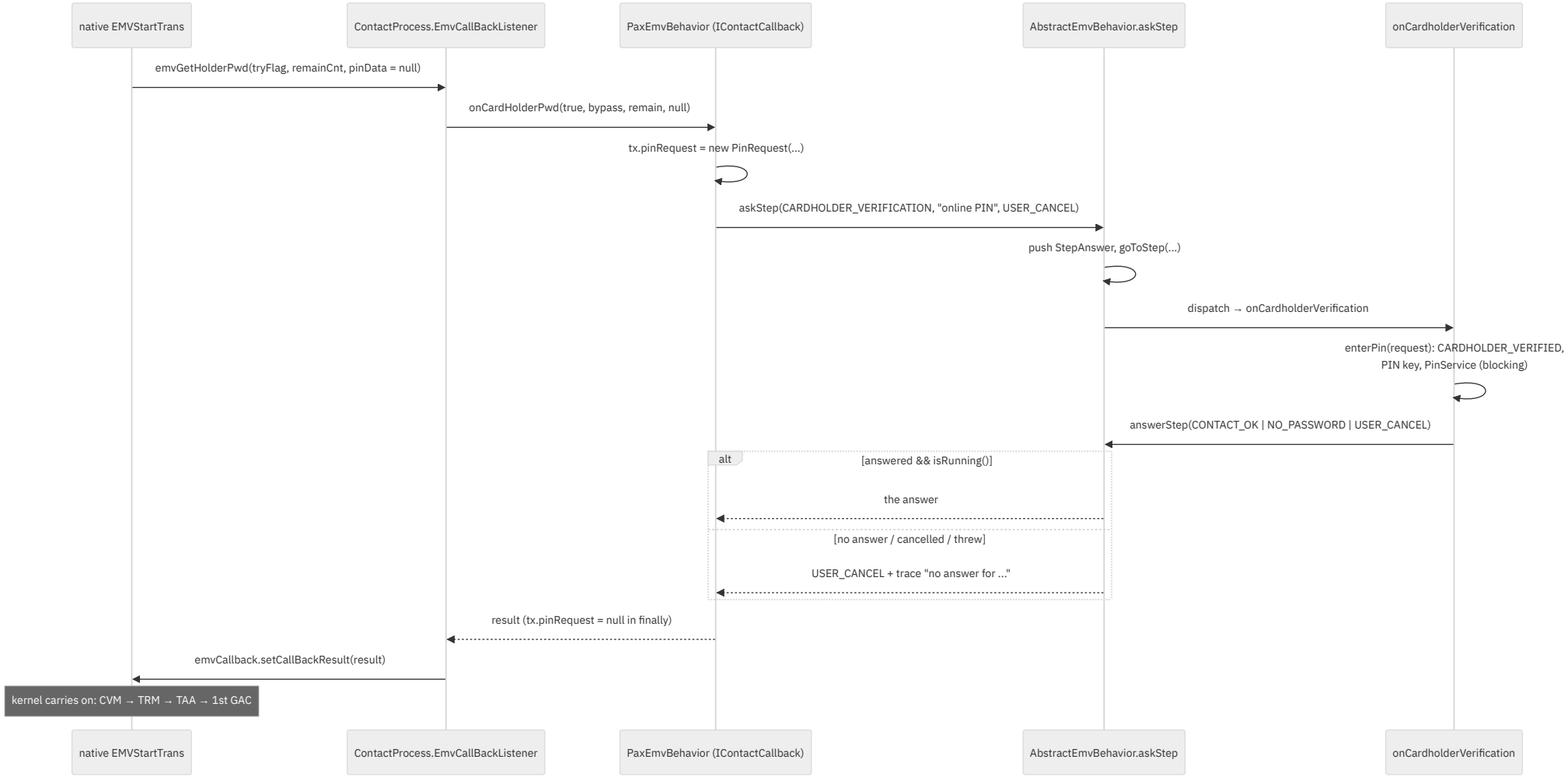
4. حين ينتظر الـ kernel شخصًا: `askStep`

أثناء استدعاء native (مثل `EMVStartTrans`) يتوقف الـ kernel ويسأل سؤالًا عبر callback: اختر التطبيق، أدخل الـ PIN. يجب أن يعود له ردٌّ دائمًا (`setCallBackResult`)، وإلا يبقى منتظرًا. الـ callback لا يفتح شاشة بنفسه؛ يسأل الخطوة المسؤولة عبر `askStep`.


① the kernel: onCardHolderPwd(...)
waits for a number
② askStep(OFFLINE_PIN_VERIFICATION, …, USER_CANCEL)
an empty slot for the answer
③ goToStep → onOfflinePinVerification
opens the PIN pad
④ answerStep(CONTACT_OK)
⑤ an answer, and still running?
yes
cancel / no answer / exception
the answer
USER_CANCEL
⑥ setCallBackResult → the kernel goes on

- قيمتها في الحالات غير الطبيعية: إلغاء، أو لا رد، أو استثناء، فيصل إلى الـ kernel "إلغاء" فورًا ولا يبقى منتظرًا، ولا يوافق أبدًا دون رد (fail closed).
- الـ callbacks التي تسأل: `onWaitAppSelect` (اختيار التطبيق)، و `onCardHolderPwd` (الـ PIN). أما callbacks البيانات (`emvSetParam` ، `emvInputAmount`) فترد داخل `ContactProcess` دون أي خطوة.

بالتفصيل (online PIN)

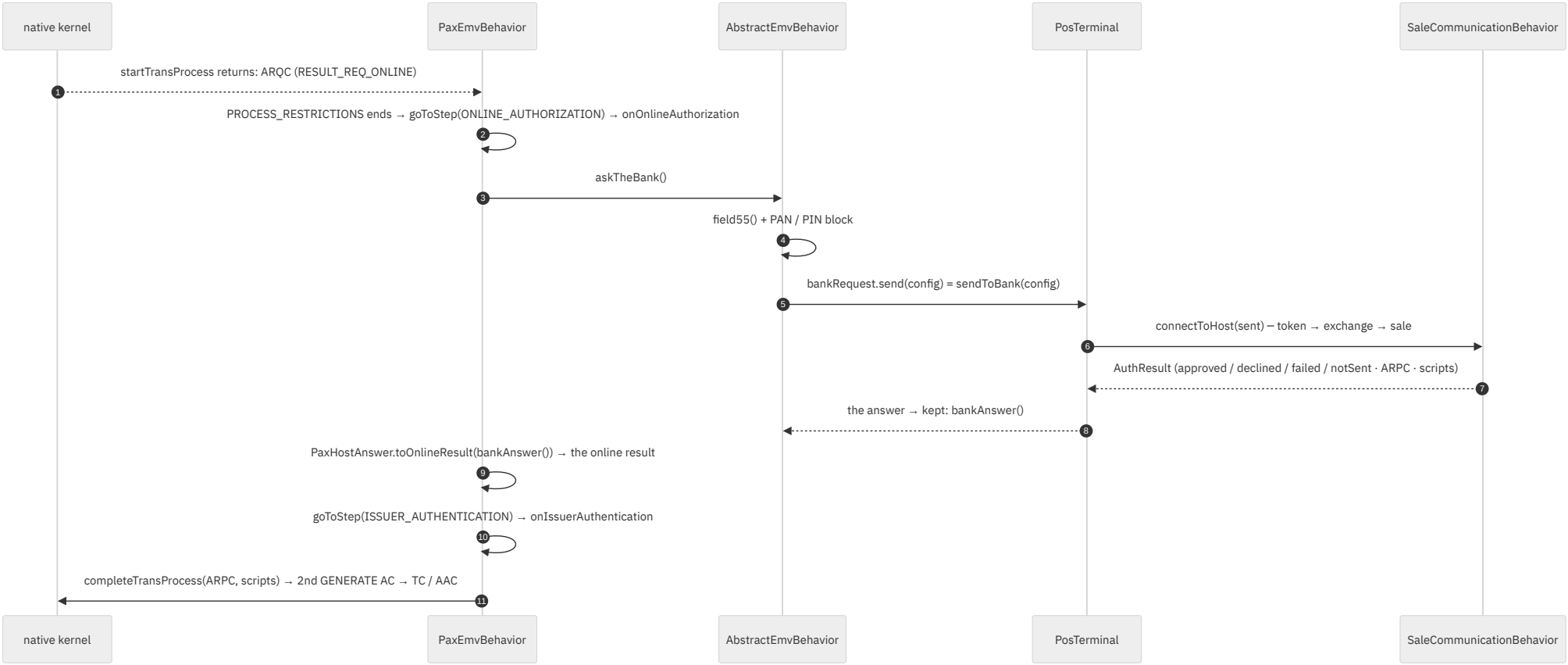

native EMVStartTrans
ContactProcess.EmvCallBackListener
PaxEmvBehavior (IContactCallback)
AbstractEmvBehavior.askStep
onCardholderVerification
emvGetHolderPwd(tryFlag, remainCnt, pinData = null)
onCardHolderPwd(true, bypass, remain, null)
tx.pinRequest = new PinRequest(...)
askStep(CARDHOLDER_VERIFICATION, "online PIN", USER_CANCEL)
push StepAnswer, goToStep(...)
dispatch → onCardholderVerification
enterPin(request): CARDHOLDER_VERIFIED,
PIN key, PinService (blocking)
answerStep(CONTACT_OK | NO_PASSWORD | USER_CANCEL)
alt
[answered && isRunning()]
the answer
[no answer / cancelled / threw]
USER_CANCEL + trace "no answer for ..."
result (tx.pinRequest = null in finally)
emvCallback.setCallBackResult(result)
kernel carries on: CVM → TRM → TAA → 1st GAC
native EMVStartTrans
ContactProcess.EmvCallBackListener
PaxEmvBehavior (IContactCallback)
AbstractEmvBehavior.askStep
onCardholderVerification

- تعمل `askStep` فقط لأن الخطوة تنتهي قبل عودة `goToStep` (§3). واستدعاء `answerStep` دون `askStep` لا يفعل شيئًا.
- الـ offline PIN: تُنشر `CARDHOLDER_VERIFICATION` للإعلان فقط، ثم `askStep(OFFLINE_PIN_VERIFICATION)`. الشاشة فقط، والـ kernel هو الذي يتحقق من الرقم مع البطاقة.
- اختيار التطبيق: `emvWaitAppSel` ← `onWaitAppSelect` ← `askStep(WAIT_ / FINAL_APPLICATION_SELECTION)` ← `answerApplicationSelection` (نافذة اختيار، أو أول تطبيق حين يوجد تطبيق واحد).
- الـ callbacks الخاصة بالبيانات تبقى في `ContactProcess` ولا تسأل أي خطوة: `emvInputAmount`، `emvSetParam` (← `resetParam(AID)`)، `emvUnknowTLVData`. وترد `certVerify` و `emvVerifyPINfailed` بـ `setCallBackResult`.

5. الطريق إلى البنك

حين يرد الكارت بـ ARQC في الـ 1st GENERATE AC، **ينتهي الاستدعاء الـ native ويرجع** بالنتيجة `RESULT_REQ_ONLINE`. بعدها كودنا هو الذي يذهب إلى البنك، ثم يعود إلى الـ kernel بالـ 2nd GENERATE AC في خطوة أخرى. ثلاث طبقات، لكل منها اسمها:

الطبقة	الاسم	ماذا تفعل
الـ EMV	`askTheBank()` (خطوة `ONLINE_AUTHORIZATIO N`)	يجهّز field 55 وبيانات البطاقة، ويسأل البنك، ويحفظ الرد: `bankAnswer()`
الجهاز	`PosTerminal.sendToBank(config)`	يسمّي الطلب (`asyncRequestId`) ويسجّله للنهاية الآمنة، وينشر `ONLINE_*`
البنك	`CommunicationBehavior.connectToHost`	HTTP عبر token ← exchange ← sale / refund


native kernel
PaxEmvBehavior
AbstractEmvBehavior
PosTerminal
SaleCommunicationBehavior
startTransProcess returns: ARQC (RESULT_REQ_ONLINE)
1
PROCESS_RESTRICTIONS ends → goToStep(ONLINE_AUTHORIZATION) → onOnlineAuthorization
2
askTheBank()
3
field55() + PAN / PIN block
4
bankRequest.send(config) = sendToBank(config)
5
connectToHost(sent) — token → exchange → sale
6
AuthResult (approved / declined / failed / notSent · ARPC · scripts)
7
the answer → kept: bankAnswer()
8
PaxHostAnswer.toOnlineResult(bankAnswer()) → the online result
9
goToStep(ISSUER_AUTHENTICATION) → onIssuerAuthentication
10
completeTransProcess(ARPC, scripts) → 2nd GENERATE AC → TC / AAC
11
native kernel
PaxEmvBehavior
AbstractEmvBehavior
PosTerminal
SaleCommunicationBehavior

كيف يعرف الـ EMV طريق البنك: `attach`

```
// BankRequest.java
public interface BankRequest { AuthResult send(TransactionConfig config); }

// AbstractEmvBehavior
private BankRequest bankRequest;
public final void attach(BankRequest bankRequest) { this.bankRequest = bankRequest; }
protected AuthResult askTheBank() { … return bankAnswer = bankRequest.send(config); }
protected final AuthResult bankAnswer() { return bankAnswer; }      // read it, never as

// PosTerminal, in its constructor, once
behavior.attach(this::sendToBank);
```

- **لماذا `attach` وليس الـ constructor؟** دائرة: `PosTerminal` يحتاج الـ behavior في الـ constructor الخاص به، والـ behavior يحتاج `PosTerminal::sendToBank` الذي لا يوجد إلا بعده. فيُنشأ الـ behavior أولًا، ثم يوصّله `PosTerminal` بالبنك مرة واحدة (setter injection). Dagger لا يحل هذه الدائرة؛ حلّه هو `attach` نفسها خلف annotations.
- **الـ EMV لا يعرف `PosTerminal`:** معه `BankRequest` فقط (Dependency Inversion). الـ field `private` و `attach` `final`، فلا يغيّر أحد الطريق.
- **نسختان من الرد، لكلٍّ مالك:** `bankAnswer()` للـ EMV (ليكمل مع البطاقة، وتُمسح مع كل محاولة)، و `OnlineRequest.answer` لـ `PosTerminal` (ليقرر التأكيد أو الإلغاء). لا يقرأ أحدهما نسخة الآخر.
- **`startOnlineProcess()` لا يناديها أحد:** هي method في واجهات PAX (`IContactCallback` / `IContactlessCallback`)، لا يناديها الـ kernel ولا كودنا. الذهاب إلى البنك هو خطوة `ONLINE_AUTHORIZATION` نفسها، وهي ترجع `bankAnswer()` بشكل PAX لو سألها أحد. تبقى لأن الواجهة تطلبها.
- **`askTheBank` تحفظ الرد في كل حالة** (`return bankAnswer = …`)، حتى حين ترجع مبكرًا (لا field 55، أو إلغاء)، فيقرأ الـ vendor السبب الحقيقي.
- الـ swipe والإدخال اليدوي لا kernel لهما: `onReadApplicationData` يذهب مباشرة إلى `ONLINE_AUTHORIZATION`، وفيها بعد `askTheBank()` يُقرَّر من `bankAnswer()` (`finishNonEmv`).

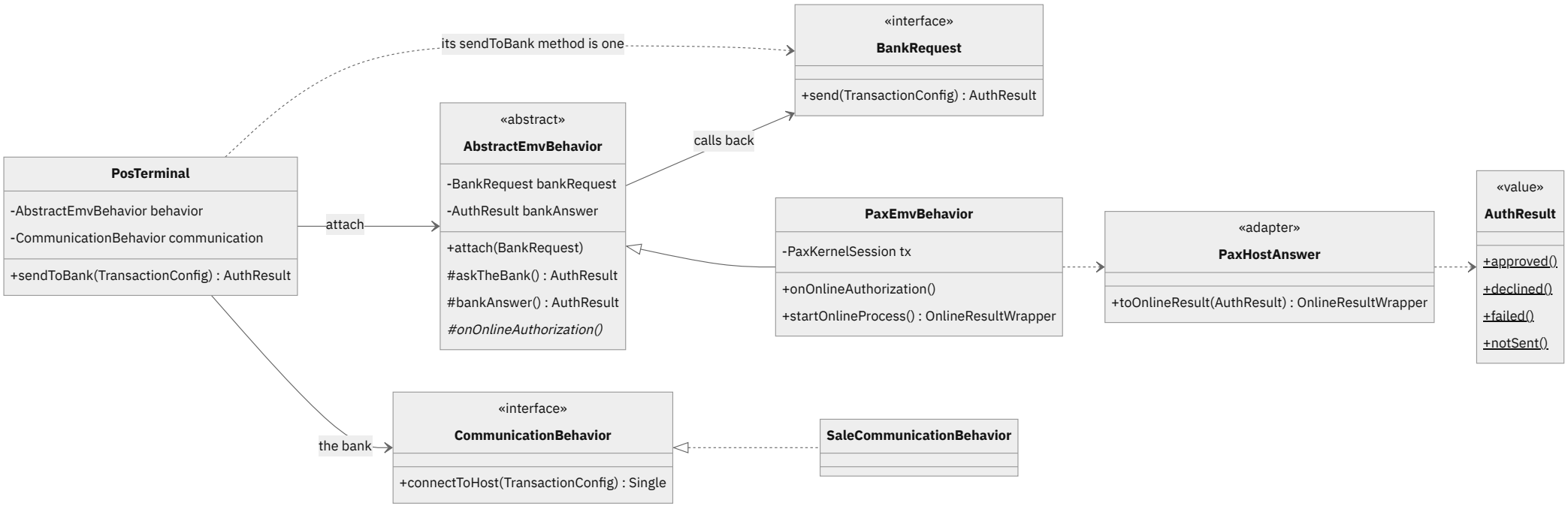
ما تراه الشاشة بعد الذهاب إلى البنك

الخطوات التي تصل إلى الشاشة بعد `ONLINE_COMPLETED`، في كل حالة:

الحالة	ما تراه الشاشة بعد `ONLINE_COMPLETED`
الـ host وافق والبطاقة أعطت TC	`APPROVED → ENDING → ACKNOWLEDGING → CLOSING_COMPLETED → COMPLETED`
الـ host وافق والبطاقة رفضت (AAC)	`DECLINED → ENDING → REVERSING → CLOSING_COMPLETED → COMPLETED`
الـ host رفض	`DECLINED → ENDING → COMPLETED`
لا رد / انتهاء المهلة (`failed`)	`ENDING → REVERSING → CLOSING_COMPLETED → ERROR`
لم يُرسل (`notSent`)	`ENDING → ERROR`

الطريق إلى البنك من الداخل: من أعطى "الرقم" لمن، وبأي مفهوم OOP

`PosTerminal` يعطي الـ behavior "رقمًا يتصل به" مرة واحدة عند إنشائه، والـ behavior يتصل به حين يرجع الـ kernel بـ ARQC، ثم يحوّل PAX الرد إلى شكله ليكمل مع البطاقة.

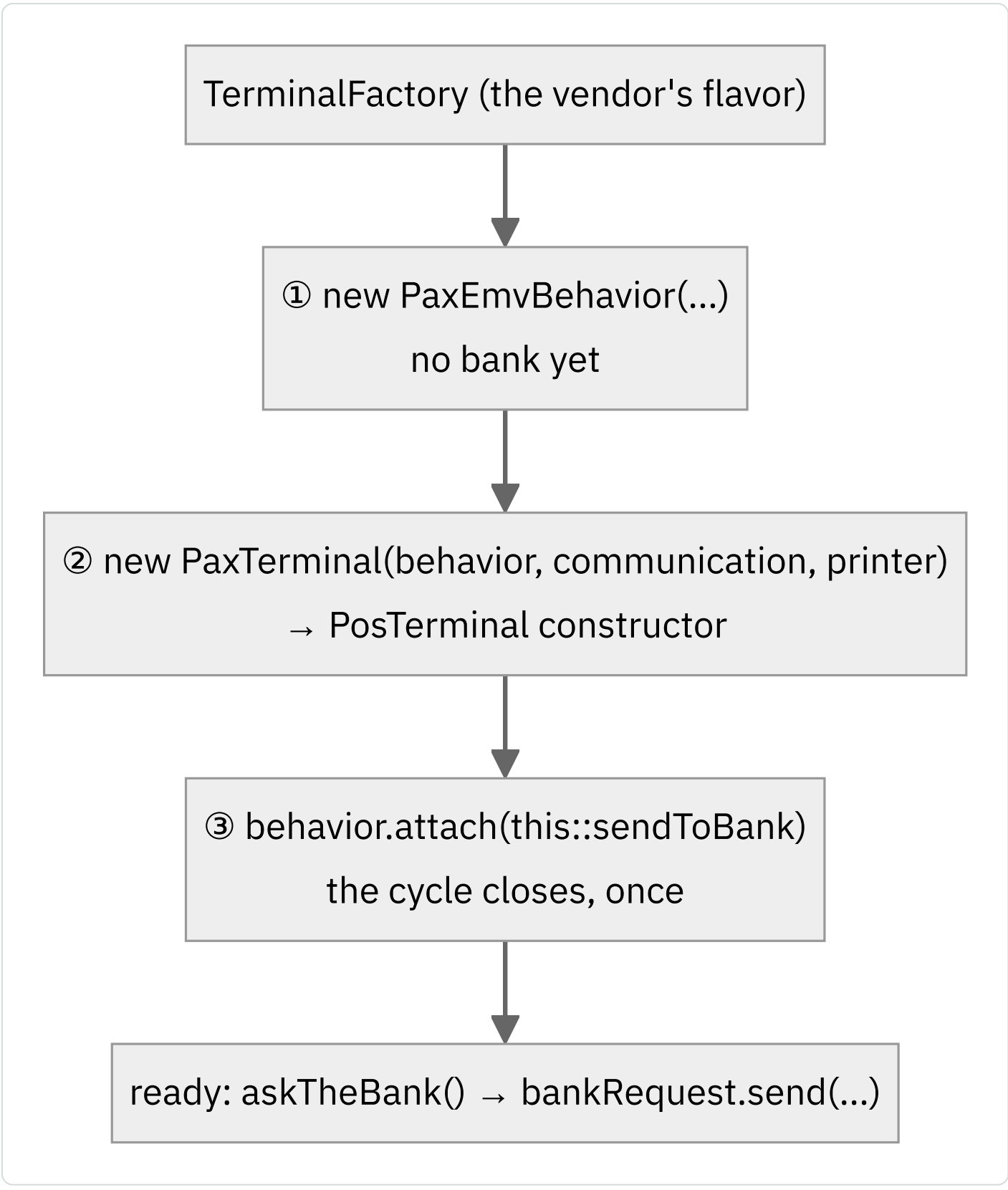

«interface»
BankRequest
+send(TransactionConfig) : AuthResult
its sendToBank method is one
«abstract»
AbstractEmvBehavior
-BankRequest bankRequest
-AuthResult bankAnswer
+attach(BankRequest)
#askTheBank() : AuthResult
#bankAnswer() : AuthResult
#onOnlineAuthorization()
calls back
PosTerminal
-AbstractEmvBehavior behavior
-CommunicationBehavior communication
+sendToBank(TransactionConfig) : AuthResult
attach
PaxEmvBehavior
-PaxKernelSession tx
+onOnlineAuthorization()
+startOnlineProcess() : OnlineResultWrapper
«adapter»
PaxHostAnswer
+toOnlineResult(AuthResult) : OnlineResultWrapper
«value»
AuthResult
+approved()
+declined()
+failed()
+notSent()
the bank
«interface»
CommunicationBehavior
+connectToHost(TransactionConfig) : Single
SaleCommunicationBehavior

المفهوم	أين في القصة	ماذا يعطينا
Callback	`attach(this::sendToBank)` ثم `bankReques` `t.send(config)`	الـ behavior لا يستدعي `PosTerminal` بالاسم؛ معه "رقم" فقط، ويتصل به حين يحتاج.
Dependency Inversion (الـ D في SOLID)	`AbstractEmvBehavior` يعتمد على الـ `B` interface `PosTerminal` `ankRequest`، لا على	طبقة الـ EMV لا تعرف شيئًا عن البنك أو الـ terminal. في الـ tests يُعطى `BankRequest` مزيف بسطر واحد.
Functional interface + method reference	`BankRequest` له method واحدة، و`this::sendT` `oBank` تكتب تنفيذها	لا حاجة لكلاس يطبّق `BankRequest`؛ الـ method الموجودة في `PosTerminal` هي التنفيذ.
Template Method	`askTheBank()` (في الـ abstract) و`onOnlineAut` `horization()` (abstract، كل vendor يملؤها)	الخطوات المشتركة (الفحوص، 55 field، الاتصال) في مكان واحد؛ الـ vendor يقرّر متى يطلبها وماذا يفعل بالرد.
Encapsulation	`private BankRequest bankRequest` و`attac` `final` `h`	لا يغيّر أحد "الرقم".
Polymorphism	`CommunicationBehavior` (التطبيق: `SaleCommu` `nicationBehavior`)، و `AbstractEmvBehavior` (PAX، Ingenico، Fake)	`PosTerminal` يتعامل مع الواجهة؛ يتغيّر البنك أو الـ vendor دون أن يتغيّر هو.
Value Object	`AuthResult`: constructor خاص، وأربع factory methods (`approved` / `declined` / `failed` / `notSent`)، ولا setters	رد البنك شيء ثابت لا يتغيّر بعد إنشائه، وبشكل واحد لا يخص vendor.
Adapter	`PaxHostAnswer.toOnlineResult(AuthResul` `t)`	يحوّل الرد العام إلى شكل PAX (`OnlineResultWrapp` `er` / `IssuerRspData`) الذي يفهمه الـ kernel. vendor آخر يكتب Adapter خاصًا به.

> كل ذلك متزامن على الـ transaction thread: `askTheBank` تنتظر `PosTerminal.sendToBank` حتى يرجع `AuthResult`، ثم يكمل الكود مع البطاقة.

لماذا `attach`، لا constructor ولا Dagger

المشكلة دائرة. `PosTerminal` يحتاج الـ behavior في الـ constructor الخاص به، والـ behavior يحتاج `PosTerminal::sendToBank`، وهذه لا توجد إلا بعد أن يُنشأ `PosTerminal`. أيهما يُنشأ أولًا لا يجد الآخر.


TerminalFactory (the vendor's flavor)
① new PaxEmvBehavior(...)
no bank yet
② new PaxTerminal(behavior, communication, printer)
→ PosTerminal constructor
③ behavior.attach(this::sendToBank)
the cycle closes, once
ready: askTheBank() → bankRequest.send(...)

الحل	ماذا يحدث	القرار
constructor لـ behavior يأخذ `BankRequest`	عند إنشاء الـ behavior لا يوجد `PosTerminal` بعد، فلا يوجد `this::sendToBank` نمرّره	غير ممكن
`PosTerminal` هو الذي يُنشئ الـ behavior	الـ core يصبح مضطرًا لمعرفة كلاسات الـ vendor (`PaxEmvBehavior`)، وهذا يكسر فصل الـ vendor	مرفوض
Dagger / Hilt	يواجه الدائرة نفسها، وحلّه `Lazy<>` أو `Provider<>` أو الحقن بعد الإنشاء: أي `attach` نفسها خلف annotations، مع Modules و Components وكود مولّد ووقت build أطول، لـ graph صغير	لا يبسّط شيئًا هنا
`attach` مرة واحدة في constructor الـ `PosTerminal`	"حقن بعد الإنشاء" (setter injection) واحد: الـ field `private` ، و `final` ، `attach` ويحدث قبل أي عملية	**المختار**

نحن نستخدم Dependency Injection فعلًا، بلا framework (manual constructor injection):
`TerminalFactory` في كل flavor يختار الـ vendor، و `PaxTerminal` يمرّر الأجزاء الثلاثة إلى `PosTerminal` : `super(new PaxEmvBehavior(kernel, pinEntry()), communication, printer)` . لا يُنشئ أي كلاس ما يحتاجه بنفسه، بل يأخذه من الخارج. Dagger / Hilt يصبح مفيدًا حين يكبر الـ graph (شاشات و ViewModels كثيرة تشترك في الـ services)، لكنه لن يلغي `attach` .

ليه `attach` مهمة: كل كلاس ماسك مسؤوليته

`attach` نوع من الـ **Dependency Injection** اسمه **setter injection**. والفكرة كلها إن كل كلاس يأخذ ما يحتاجه من الخارج، ولا يصنعه بنفسه، فيبقى مسؤولًا عن شغله فقط.

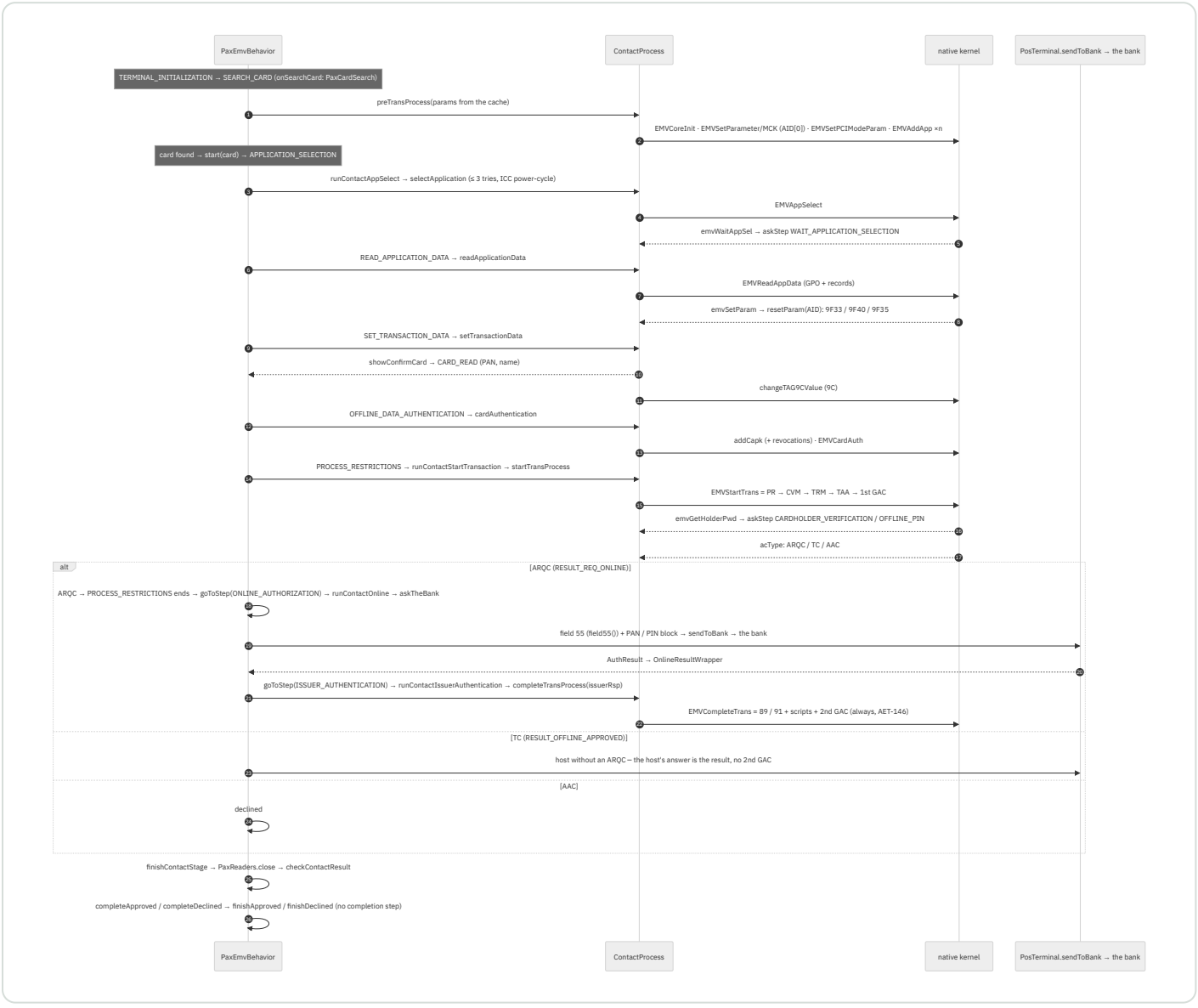
- **بدون DI:** كان الـ EMV سيضطر أن يعرف `PosTerminal` بنفسه، أو أن يتصل بالبنك بنفسه، فيحمل مسؤولية ليست له.
- **مع DI:** يأخذ الـ EMV "طريق البنك" من الخارج (`BankRequest`) ويستخدمه فقط. لا يعرف من على الطرف الآخر، ولا كيف يتكلم البنك.

الكلاس	شغله	ما يأخذه من الخارج
`AbstractEmvBehavior`	البطاقة والـ kernel	`BankRequest` (بـ `attach`)
`PosTerminal`	العملية والجهاز	الـ behavior والـ communication والـ printer (في الـ constructor)
`SaleCommunicationBehavior`	الكلام مع البنك	مفاتيح وإعدادات الـ onboarding

المفهوم	معناه	دوره هنا
Single Responsibility	كل كلاس له شغلة واحدة	**الهدف**
Dependency Injection	كيف يصل كل كلاس إلى ما يحتاجه: من الخارج بدل أن يصنعه (constructor مثل `PosTerminal` ، أو `atta` `ch` مثل `bankRequest`)	أداة
Dependency Inversion	ما يُحقَن يكون interface (`BankRequest`) لا كلاسًا محددًا	أداة: لهذا لا يعرف الـ EMV `PosTerminal` إطلاقًا

6. مسار الـ contact (بترتيب الـ kernel نفسه)

كل مرحلة = استدعاء native واحد = `EmvStep` واحدة. كل `runContact*` تستدعي الـ kernel: إن نجحت تنتقل بـ `advanceContactStage(NEXT)`، وإلا تنهي المسار بـ `finishContactStage()` (تحرير الـ listener، وإغلاق القارئات، وحساب النتيجة).



#	EmvStep	الدالة في PAX	native	يسأل شخصًا؟
0	`TERMINAL_INITIALIZATION`	`preTrans` → `prepareKernelProcess`	`EMVCoreInit`، `EMVAddApp`	—
1	`SEARCH_CARD`	`PaxCardSearch` → `onSearchCard`	polling	—
2	`APPLICATION_SELECTION`	`runContactAppSelect`	`EMVAppSelect`	اختيار التطبيق
5	`READ_APPLICATION_DATA`	`runContactReadAppData`	`EMVReadAppData` (GPO + records)	—
6	`SET_TRANSACTION_DATA`	`runContactSetTransactionData`	9C · `CARD_READ`	—
7	`OFFLINE_DATA_AUTHENTICATION`	`runContactCardAuth`	`EMVCardAuth`، `EMVAddCAPKth`	—
8	`PROCESS_RESTRICTIONS`	`runContactStartTransaction`، ثم 13 إن ذهب الكارت إلى البنك، وإلا نهاية المرحلة	`EMVStartTrans` = PR + CVM + TRM + TAA + 1st GAC	الـ PIN
13	`ONLINE_AUTHORIZATION`	`onOnlineAuthorization` → `runContactOnline`، ثم 14 دائمًا	(البنك) —	—
14	`ISSUER_AUTHENTICATION`	`runContactIssuerAuthentication`، ثم نهاية المرحلة	`EMVCompleteTrans` = ARPC + scripts + 2nd GAC	—

- **متى يذهب الطلب إلى البنك:** حين يرد الكارت بـ ARQC. الاسترداد يطلب ARQC كالبيع. الاستثناء: TC على جهاز online فقط، فيُرسل دون ARQC ويكون رد البنك هو النتيجة. AAC = رفض. (`RESULT_OFFLINE_APPROVED`)
- **كل خطوة تعمل عملها ثم تسلّم التالية:** `PROCESS_RESTRICTIONS` → `ONLINE_AUTHORIZATION` → `ISSUER_AUTHENTICATION`، كل منها بالـ hook الخاص بها. البنك والـ issuer لم يعودا داخل الـ hook الخاص بـ `PROCESS_RESTRICTIONS`. و `finishContactStage` في آخر خطوة تعمل، أو حيث يتوقف المسار. الإلغاء أثناء انتظار البنك يُغلق المرحلة دون 2nd GAC.
- **الـ 2nd GENERATE AC دائمًا** بعد البنك (AET-146)، حتى لو رفض البنك أو لم يرد.
- في الـ contact، الـ scripts تعمل داخل `EMVCompleteTrans` نفسه، فـ `SCRIPT_PROCESSING` هنا مجرد عنوان، وليست مرحلة منفصلة.
- ⚠ `TERMINAL_RISK_MANAGEMENT` و `TERMINAL_ACTION_ANALYSIS` لا تُنشران: لا يوجد لهما استدعاء native منفصل (داخل `EMVStartTrans`).

7. النتيجة والنهاية الآمنة

الـ EMV يبلّغ بالنتيجة (`publish(APPROVED / DECLINED)` → `finishApproved` / `finishDeclined`، و`finishError` ← `fail`)، والـ `Transaction` هي التي تنهي العملية. وقبل أن يصبح الجهاز متاحًا، يُغلق كل طلب أُرسل إلى البنك: **تأكيد** (acknowledgement) أو **إلغاء** (reversal).

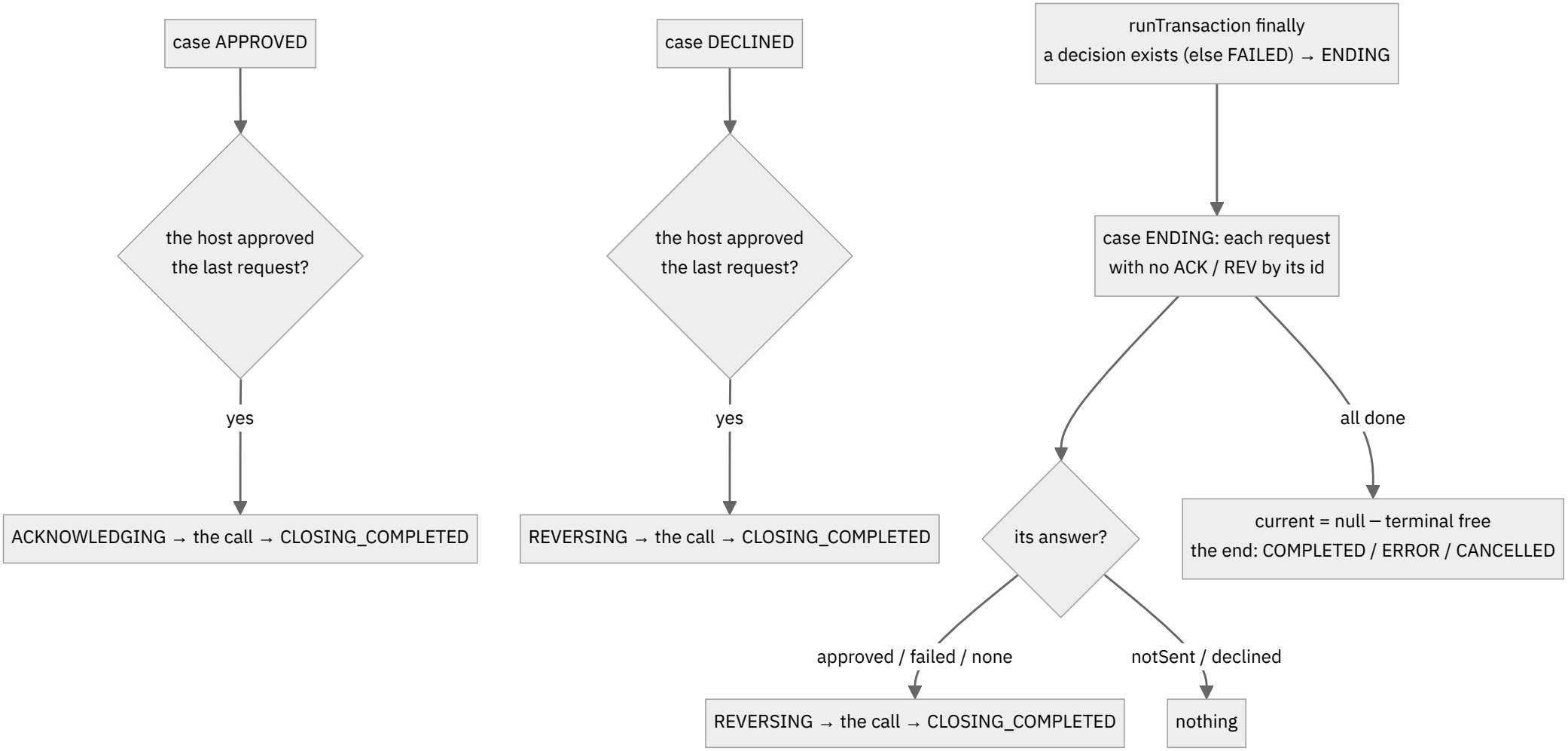
- **القرار نفسه يُغلق آخر طلب:** `case APPROVED` ← تأكيد، و`case DECLINED` ← إلغاء، بشرط أن البنك وافق على آخر طلب (إن رفض البنك فلا شيء). في هذه اللحظة رد البنك معروف، والبطاقة قررت، والـ reader مغلق.
- **`ENDING` يُغلق الباقي:** كل طلب لا توجد له `ACKNOWLEDGING` / `REVERSING` بالـ id نفسه في الخطوات يُلغى إن كان البنك ربما وافق (موافقة، `failed`، لا رد). هذا يغطي الإلغاء (قد يأتي والبنك ما زال يرد)، و`FAILED`، وطلبات محاولة سابقة. ثم يتحرر الجهاز وتُسلَّم النهاية.

البنك	البطاقة في الـ 2nd GAC	النتيجة	عند البنك بعدها
وافق	TC	APPROVED → COMPLETED	تأكيد
وافق	AAC	DECLINED → COMPLETED	إلغاء
رفض	—	DECLINED → COMPLETED	لا شيء
لا رد / انتهت المهلة (`failed`)	AAC	ERROR	إلغاء (ربما وافق)
لم يُرسل (`notSent`)	AAC	ERROR	لا شيء (لم يصل)

بالتفصيل: ماذا يقول الـ kernel

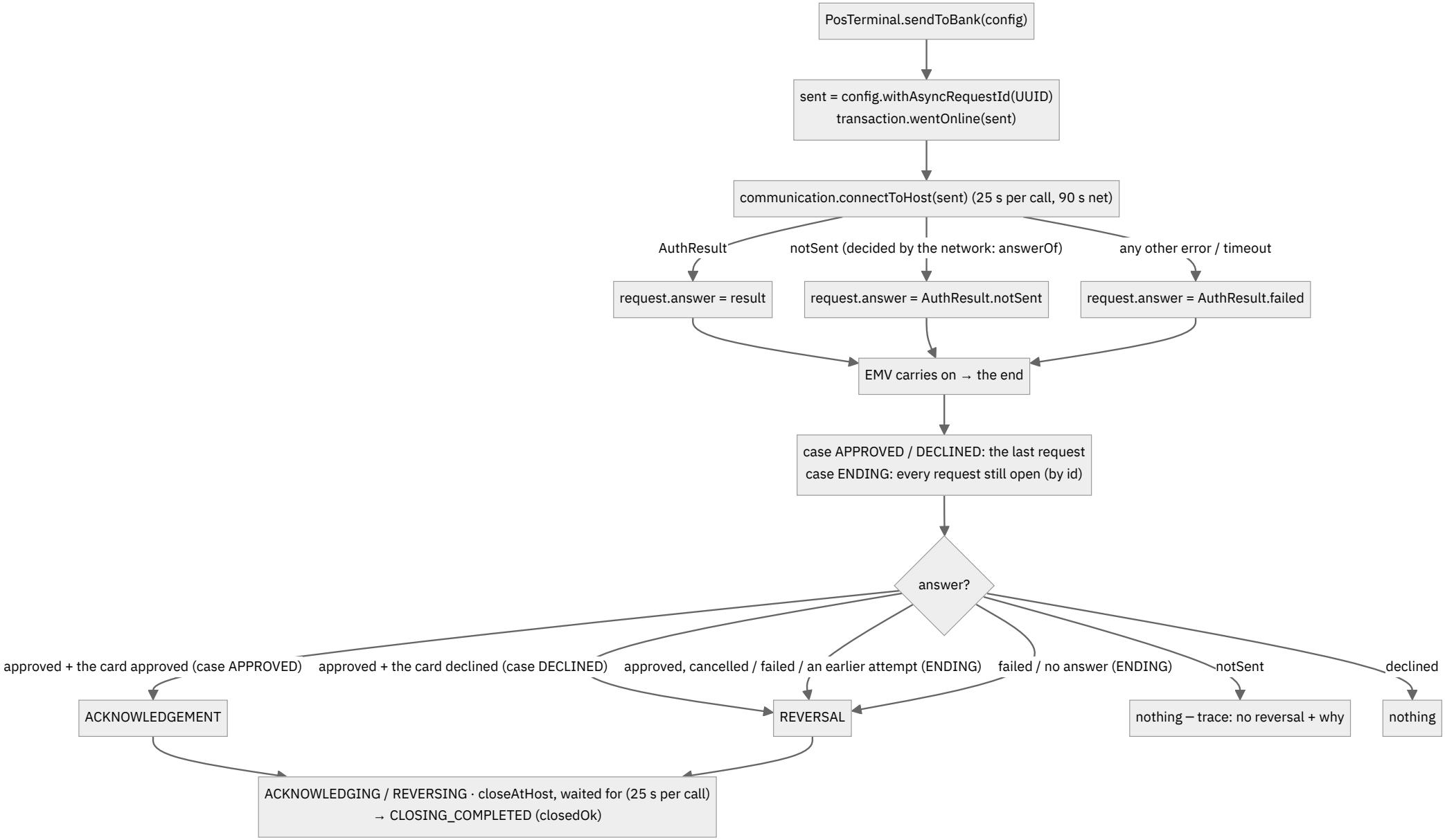
الـ host	البطاقة في 2nd GAC	`TransResultEnum`	النهاية	مع النهاية
APPROVE	TC	`ONLINE_APPROVED`	APPROVED → COMPLETED	acknowledge
APPROVE	غير ذلك	`ONLINE_CARD_DENIED`	DECLINED → COMPLETED	reverse
FAILED (لا رد / انتهاء المهلة / لم يُرسل)	أي نتيجة (الجهاز يطلب AAC)	`ONLINE_FAILED` / `FAILED` / `_CARD_APPROVED`	ERROR "Online failed: <<step>>: ..."	reverse، إلا إذا كانت `notSent`
DENIAL	—	`ONLINE_DENIED`	DECLINED → COMPLETED	لا شيء

> ⚠ **مخاطر مفتوحة:** الإلغاء أثناء انتظار رد الـ host ينهي العملية بـ CANCELLED حتى لو وافق الـ host وأعطت البطاقة TC، وفي هذه الحالة يُرسل إلغاء (reversal) لذلك الطلب. يُرسل الإلغاء مرة واحدة: إذا فشل يُسجَّل في الـ log والـ trace فقط، دون سجل محفوظ لإعادة الإرسال (BACKLOG 0.15).


case APPROVED
the host approved the last request?
yes
ACKNOWLEDGING → the call → CLOSING_COMPLETED
case DECLINED
the host approved the last request?
yes
REVERSING → the call → CLOSING_COMPLETED
runTransaction finally
a decision exists (else FAILED) → ENDING
case ENDING: each request with no ACK / REV by its id
all done
current = null – terminal free
the end: COMPLETED / ERROR / CANCELLED
its answer?
approved / failed / none
notSent / declined
REVERSING → the call → CLOSING_COMPLETED
nothing

كيف يُقرَّر التأكيد أو الإلغاء لكل طلب

كل طلب أُرسل يُسجَّل لحظة إرساله (`wentOnline`) ومعه رده و id. القرار يُغلق آخر طلب، و `ENDING` يُغلق الباقي، والـ id هو الذي يربط بينهما:


PosTerminal.sendToBank(config)
sent = config.withAsyncRequestId(UUID)
transaction.wentOnline(sent)
communication.connectToHost(sent) (25 s per call, 90 s net)
AuthResult
notSent (decided by the network: answerOf)
any other error / timeout
request.answer = result
request.answer = AuthResult.notSent
request.answer = AuthResult.failed
EMV carries on → the end
case APPROVED / DECLINED: the last request
case ENDING: every request still open (by id)
answer?
approved + the card approved (case APPROVED)
approved + the card declined (case DECLINED)
approved, cancelled / failed / an earlier attempt (ENDING)
failed / no answer (ENDING)
notSent
declined
ACKNOWLEDGEMENT
REVERSAL
nothing – trace: no reversal + why
nothing
ACKNOWLEDGING / REVERSING · closeAtHost, waited for (25 s per call)
→ CLOSING_COMPLETED (closedOk)

من يقرر `notSent` ؟ الـ network وحده

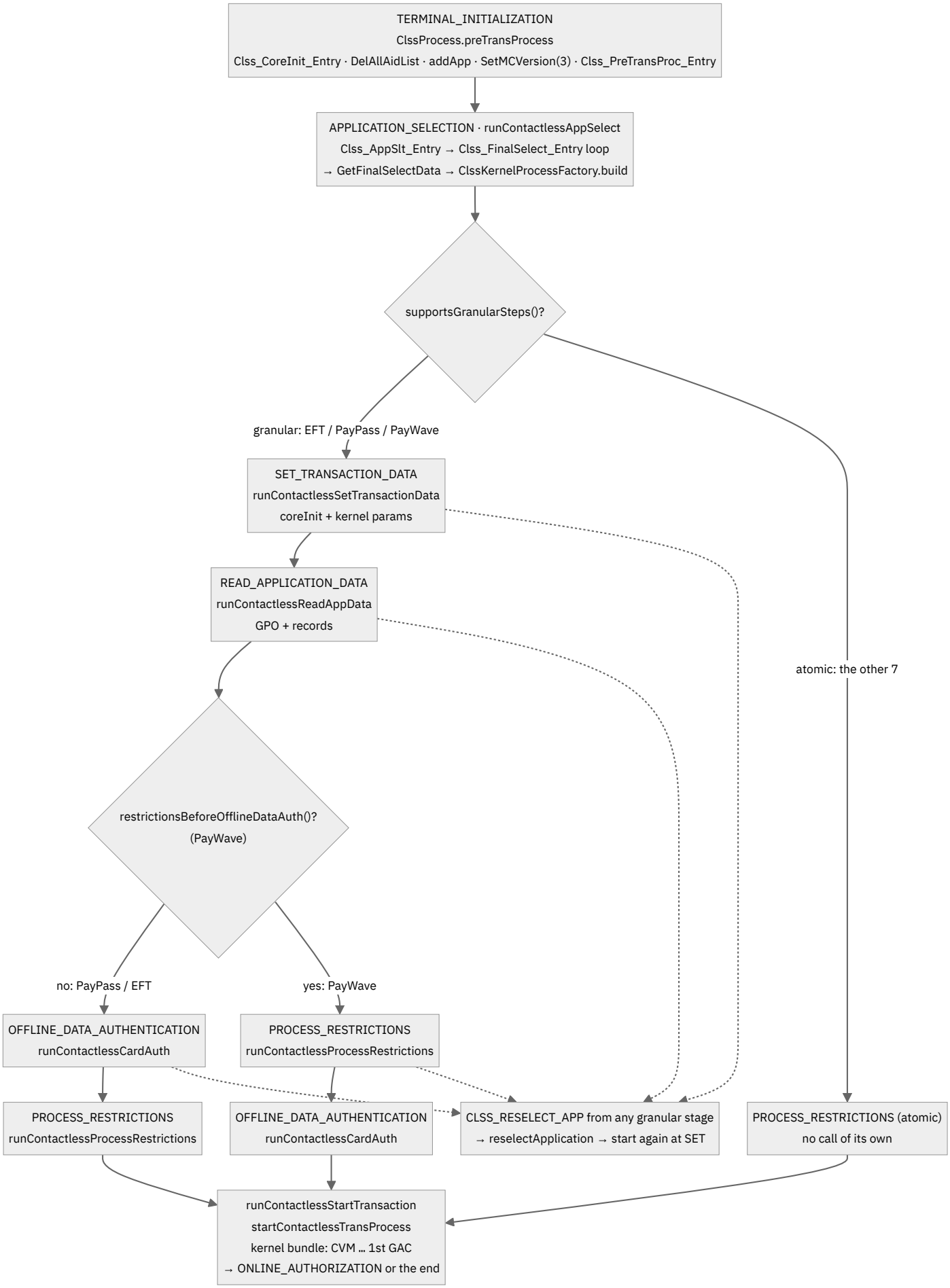
أين فشل	النتيجة	إلغاء؟
قبل الإرسال: لا مرجع للاسترداد، لا EMV result / مفتاح / token، فشل التشفير	`notSent`	لا
`Token refresh: …` أو `Exchange: …`	`notSent`	لا
`Sale:` لا اتصال (`ConnectException` / `Unknow` `nHost`) أو HTTP 401 / 403	`notSent`	لا
`Sale:` انتهاء المهلة، انقطاع، 4xx، 5xx، آخر، envelope تالف	`failed`	نعم

المهلة	القيمة	ملاحظة
كل نداء دفع (OkHttp `callTimeout`)	25 ث	token + exchange + sale ≤ 75 ث؛ للدفع فقط، لا لتنزيل الـ params
`PosTerminal.sendToBank`	90 ث	شبكة أمان فوق الـ 75
`closeAtHost` (تأكيد / إلغاء)	60 ث	شبكة أمان؛ الحد الفعلي 25 ث للنداء

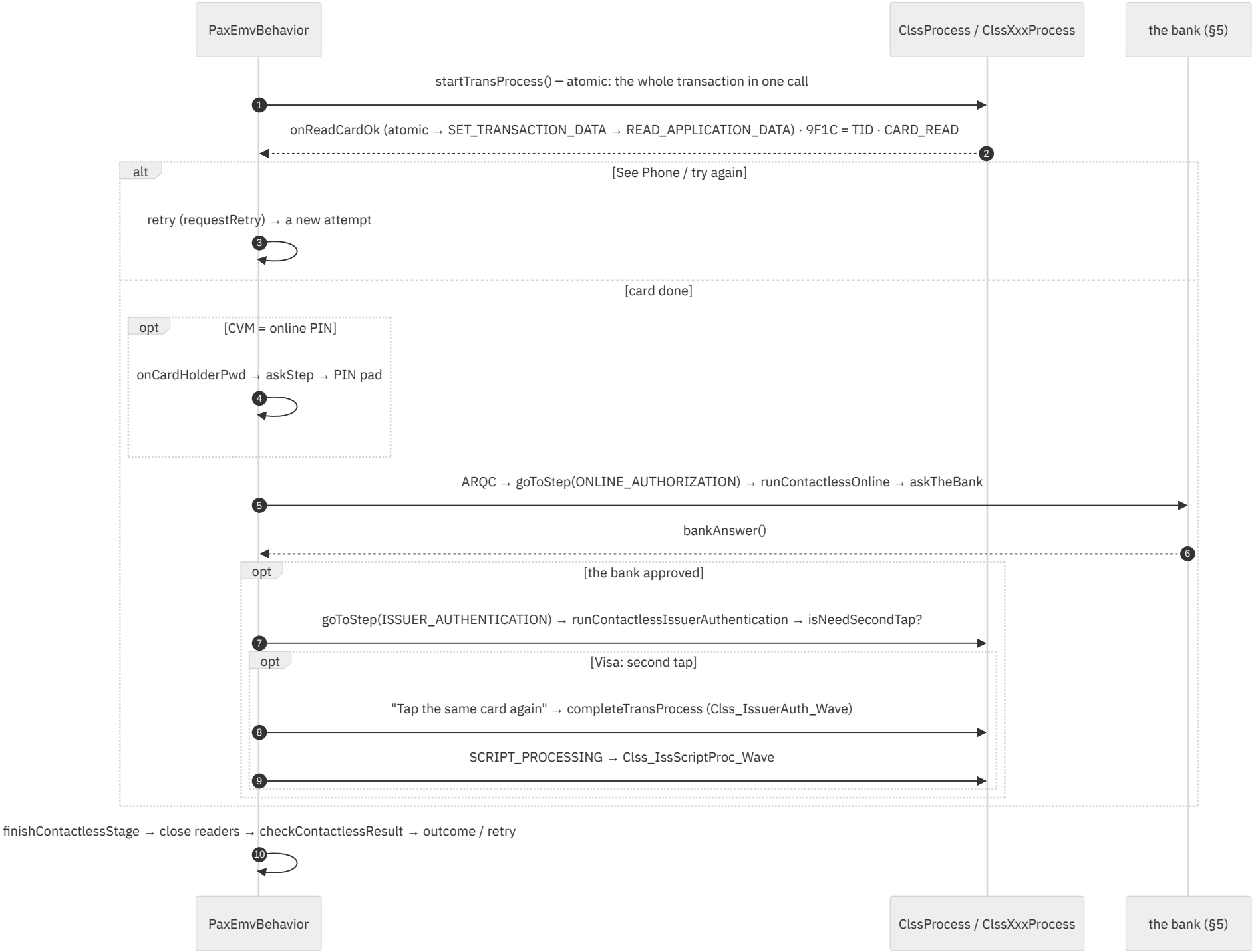
- نداءات الدفع تستخدم الـ adapter المتزامن (`createSynchronous()`)، وإلا يبتلع Retrofit الخطأ حين يقطع `callTimeout` النداء، فيبقى الجهاز مشغولًا للأبد (أُصلح في `1a97dee`).
- ⚠ التأكيد / الإلغاء يُرسل مرة واحدة: إن فشل يُسجَّل في الـ trace فقط، بلا سجل محفوظ لإعادة الإرسال (BACKLOG 0.15).
- يظهر اسم المرحلة في رسالة الـ ERROR وفي الـ trace: `Online Failed: Exchange: HTTP 403 Forbidden` .
- الـ trace (tag `HOST`): `Acknowledgement <id> sent` أو `… failed — not retried: <why>`، `REVERSAL for request <id>: approved, not completed`، `REVERSAL …: no usable host`، `answer — Sale: timeout`، `no reversal …: the host could not have approved it — Exchange: HTTP 403 Forbidden`.
- لا يتغير شيء بالنسبة للبطاقة: `failed` و `notSent` كلاهما `RESULT_ONLINE_FAILED` (2nd GENERATE AC بـ AAC). الفرق في الإلغاء فقط.
- ⚠ لم يُختبر على الجهاز ولا مع الـ host. احتياج الإلغاء والتأكيد إلى `mToken` افتراض.

8. مسار الـ contactless

طريقة التشغيل نفسها و `askStep` نفسها، لكن الـ kernel يُختار بعد اختيار التطبيق (`ClssKernelProcessFactory`). نوعان: **granular** (PayPass و PayWave و EFT) ولكل مرحلة استدعاء native مستقل، و**atomic** (Amex و JCB و Mir و PBOC و Pure و RuPay و DPAS) وفيها استدعاء واحد يعمل كل شيء.



بعد المراحل: `startContactlessTransProcess`


PaxEmvBehavior
ClssProcess / ClssXxxProcess
the bank (§5)
startTransProcess() — atomic: the whole transaction in one call
1
onReadCardOk (atomic → SET_TRANSACTION_DATA → READ_APPLICATION_DATA) · 9F1C = TID · CARD_READ
2
alt
[See Phone / try again]
retry (requestRetry) → a new attempt
3
[card done]
opt
[CVM = online PIN]
onCardHolderPwd → askStep → PIN pad
4
ARQC → goToStep(ONLINE_AUTHORIZATION) → runContactlessOnline → askTheBank
5
bankAnswer()
6
opt
[the bank approved]
goToStep(ISSUER_AUTHENTICATION) → runContactlessIssuerAuthentication → isNeedSecondTap?
7
opt
[Visa: second tap]
"Tap the same card again" → completeTransProcess (Clss_IssuerAuth_Wave)
8
SCRIPT_PROCESSING → Clss_IssScriptProc_Wave
9
finishContactlessStage → close readers → checkContactlessResult → outcome / retry
10
PaxEmvBehavior
ClssProcess / ClssXxxProcess
the bank (§5)

- **الترتيب يتبع الـ kernel، لا الأرقام:** في الـ atomic تُعلَن `READ_APPLICATION_DATA` و `SET_TRANSACTION_DATA` من الـ callbacks في اللحظة التي يقول فيها الـ kernel إنها حدثت، ولا تُعاد القراءة (الـ hooks لا تعمل شيئًا للـ atomic).
- إذا لم يوافق البنك، فلا issuer authentication ولا 2nd GENERATE AC: البطاقة غادرت الحقل.
- ⚠ رسالة الـ tap الثاني لا تنتظر البطاقة قبل `Clss_FinalSelect_Entry` (BACKLOG 1.4).

EmvStep	PayPass (`ClssPayPassProcess`)	PayWave (`ClssPayWaveProcess`)	EFT (`ClssEFTProcess`)
`SET_TRANSACTION_DATA`	`Clss_CoreInit_MC`، `SetParam_M` + `setPayPass`، `C_Param` (DF8117/18/19/1F، limits) FinalSelectData	`CoreInit_Wave`، ReaderParam + (`aucTmCap` = 9F33) `SetVisaAi` + `dParam_Wave`، `SetTransData_Wav e`	`CoreInit_EFT` + ReaderParam + + `SetTransData_EFT` + AidParam
`READ_APPLICATION_DATA`	`Clss_InitiateApp_MC` + `Clss_R` `eadData_MC`	`Clss_Proctrans_Wave` (+ GPO (records + AC type	`Clss_InitApp_EFT` + `Clss_Read` `AppData_EFT`
الترتيب بعدها	ODA ثم PR	**PR ثم ODA**	ODA ثم PR
`OFFLINE_DATA_AUTHENTICATION`	تحميل CAPK و revocation فقط (الـ ODA داخل `TransProc`)	`Clss_CardAuth_Wave` (عند TC وليس استردادًا، أو حسب TTQ)	`PreTAAProc` + CAPK + `Clss_Car` `dAuth_EFT`
`PROCESS_RESTRICTIONS`	لا يوجد استدعاء	`Clss_ProcRestric_Wave`	`Clss_ProcRestrict_EFT`
الـ bundle	`Clss_TransProc_MC_MChip` / `_M` = `ag` ODA+PR+CVM+TRM+TAA+GAC، ثم see phone	`genTransResult` : `Clss_GetCvmT` AC type + `ype_Wave`	`Clss_CVMProc_EFT` + `Clss_Post` `TAAProc_EFT`
2nd tap / completion	`completeTransProcess`	`Clss_IssuerAuth_W` + FinalSelect `ave`	`Clss_CompleteOnlineTrans_EFT` ⚠ لا يُستدعى
`SCRIPT_PROCESSING`	default	`Clss_IssScriptProc_Wave`	default

النتيجة: `checkContactlessResult`

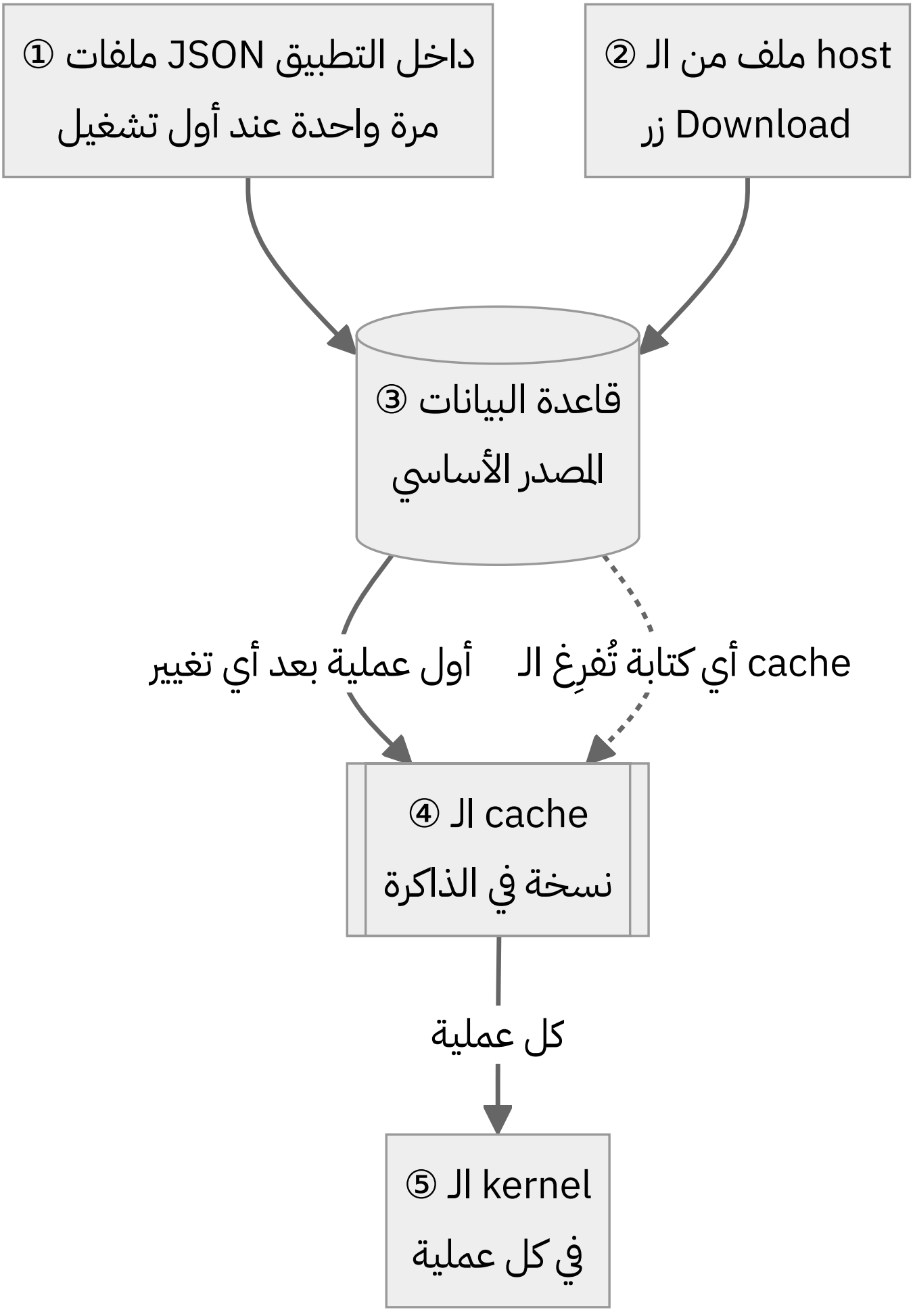
الحالة	الدالة	النهاية
`RESULT_CLSS_SEE_PHONE` (أي ret)	`seePhone` (+ `clsLastNeedSeePhone`)	إعادة المحاولة بالـ CONTACTLESS، وبعد 3 مرات رفض
`RESULT_TRY_AGAIN` (أي ret)	`tryAgain`	إعادة المحاولة بالطريقة نفسها، وبعد 3 مرات رفض
`ONLINE_APPROVED` + OK	`onlineApproved`	APPROVED
`OFFLINE_APPROVED` + OK	`offlineApproved`	"ERROR "Online Failed
OK + غير ذلك	`offlineDenied(CLSS_DECLINE)`	DECLINED
`TRY_ANOTHER_INTERFACE` / `CLSS_USE_CONTAC T`	`tryAnotherInterface`	إعادة المحاولة بالـ CHIP
`ONLINE_DENIED` / `ONLINE_CARD_DENIED` / `OF FLINE_DENIED`	كما في الـ contact	DECLINED
`ONLINE_FAILED`	`onlineFailed`	ERROR

IContactlessCallback	من يستدعيها	ماذا تفعل
onReadCardOk	الـ kernel بعد القراءة	atomic: goToStep(SET_TRANSACTION_DATA) ثم goToStep(READ_APPLICATION_DATA) ؛ APPL ICATION_SELECTED؛ 9F1C؛ CARD_READ . يأتي قبل أن يقرر الكارت، فيظهر CARD_READ أيضًا قبل See Phone أو الرفض.
confirmCard	لا أحد (الـ kernel لا يستدعيها)	ترجع CONTACT_OK فقط؛ باقية لأن الواجهة تطلبها
onWaitAppSelect / onCardHolderPwd	مشتركة مع الـ contact	askStep
startOnlineProcess	لا أحد	ترجع bankAnswer() بشكل PAX فقط؛ الذهاب إلى البنك هو ONLINE_AUTHORIZATION
onDetect2ndTap	مرحلة الإكمال	رسالة + SEARCH_CARD (للعرض فقط، دون بحث ثانٍ)
needSeePhone	الـ kernel في الـ tap التالي (Amex / Mir / Pure)	تعيد clsLastNeedSeePhone ثم تصفّرها
onRemoveCard	PayPass / PayWave	تسجيل في الـ log فقط

9. الـ EMV params: مصدرها، ومكان حفظها، ومتى تصل إلى الـ kernel

ما هي الـ params؟ هي القواعد التي يحتاجها الـ kernel ليتخذ قراره في كل عملية: البطاقات المقبولة (الـ AIDs)، والمفاتيح التي يُتحقَّق بها من أن البطاقة أصلية (الـ CAPKs)، والحدود المالية (مثل المبلغ الذي يُطلب بعده PIN في الدفع اللاتلامسي). بدونها لا يستطيع الـ kernel العمل.

الصورة العامة


① ملفات JSON داخل التطبيق
مرة واحدة عند أول تشغيل
② ملف من الـ host
زر Download
③ قاعدة البيانات
المصدر الأساسي
أول عملية بعد أي تغيير
أي كتابة تُفرِغ الـ cache
④ الـ cache
نسخة في الذاكرة
كل عملية
⑤ الـ kernel
في كل عملية

دورة حياة الـ params في 5 خطوات

1. **عند أول تشغيل للتطبيق:** تُملأ قاعدة البيانات من ملفات JSON المرفقة بالتطبيق، حتى يكون هناك ما يعمل به. يحدث ذلك مرة واحدة فقط.
2. **عند الضغط على Download:** يرسل الـ host ملفًا (XML للـ contact وآخر للـ contactless)، فيُقرأ ويُكتب في قاعدة البيانات بدلًا من القديم.
3. **قاعدة البيانات هي المصدر الأساسي:** القيم الفعلية للـ params موجودة فيها.
4. **الـ cache نسخة سريعة:** أول عملية بعد أي تغيير تقرأ قاعدة البيانات كاملة مرة واحدة وتحتفظ بها في الذاكرة، والعمليات التالية تأخذ من الذاكرة مباشرة.
5. **الـ kernel لا يحتفظ بشيء:** في **كل** عملية تُحمَّل إليه الـ params من الـ cache من جديد، وتُمسح منه بانتهاء العملية.

مثال

غيّر الـ host حدّ طلب الـ PIN في الدفع اللاتلامسي من 600 إلى 1000 جنيه:

- **Download:** أصبحت القيمة في قاعدة البيانات 1000، وأُفرِغ الـ cache.
- **أول عملية بعدها:** يُعاد ملء الـ cache من قاعدة البيانات (1000)، ويستلمها الـ kernel.
- **عملية tap بمبلغ 800 جنيه:** تتم دون طلب PIN ✓

ماذا تعرض شاشة EMV Params؟

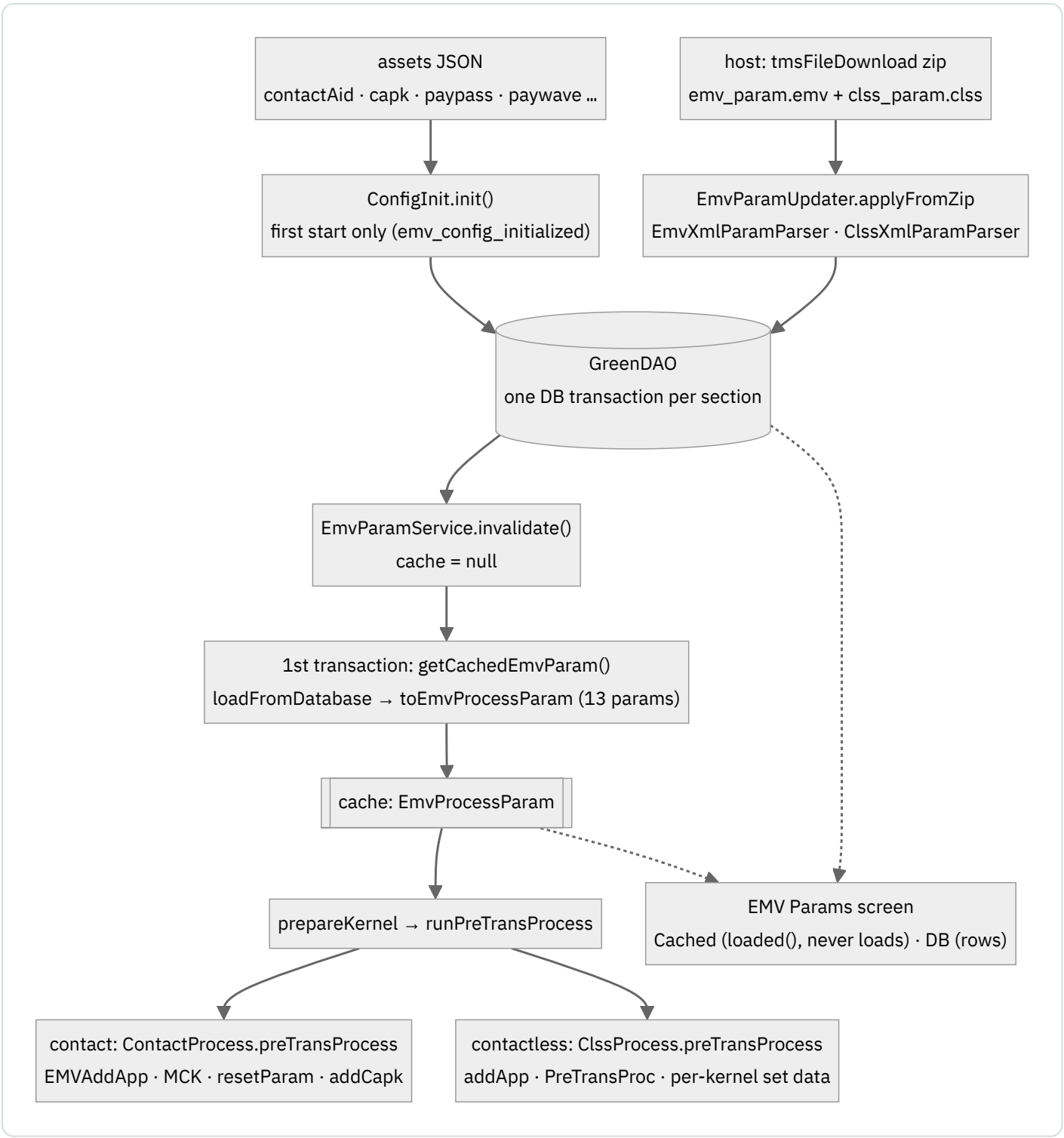
- **DB:** القيم المحفوظة (المصدر الأساسي).
- **Cached:** القيم التي سيستلمها الـ kernel الآن. إن كانت فارغة فمعنى ذلك أنه لم تتم أي عملية منذ آخر تغيير، وليس أن هناك خطأ.

قواعد مهمة

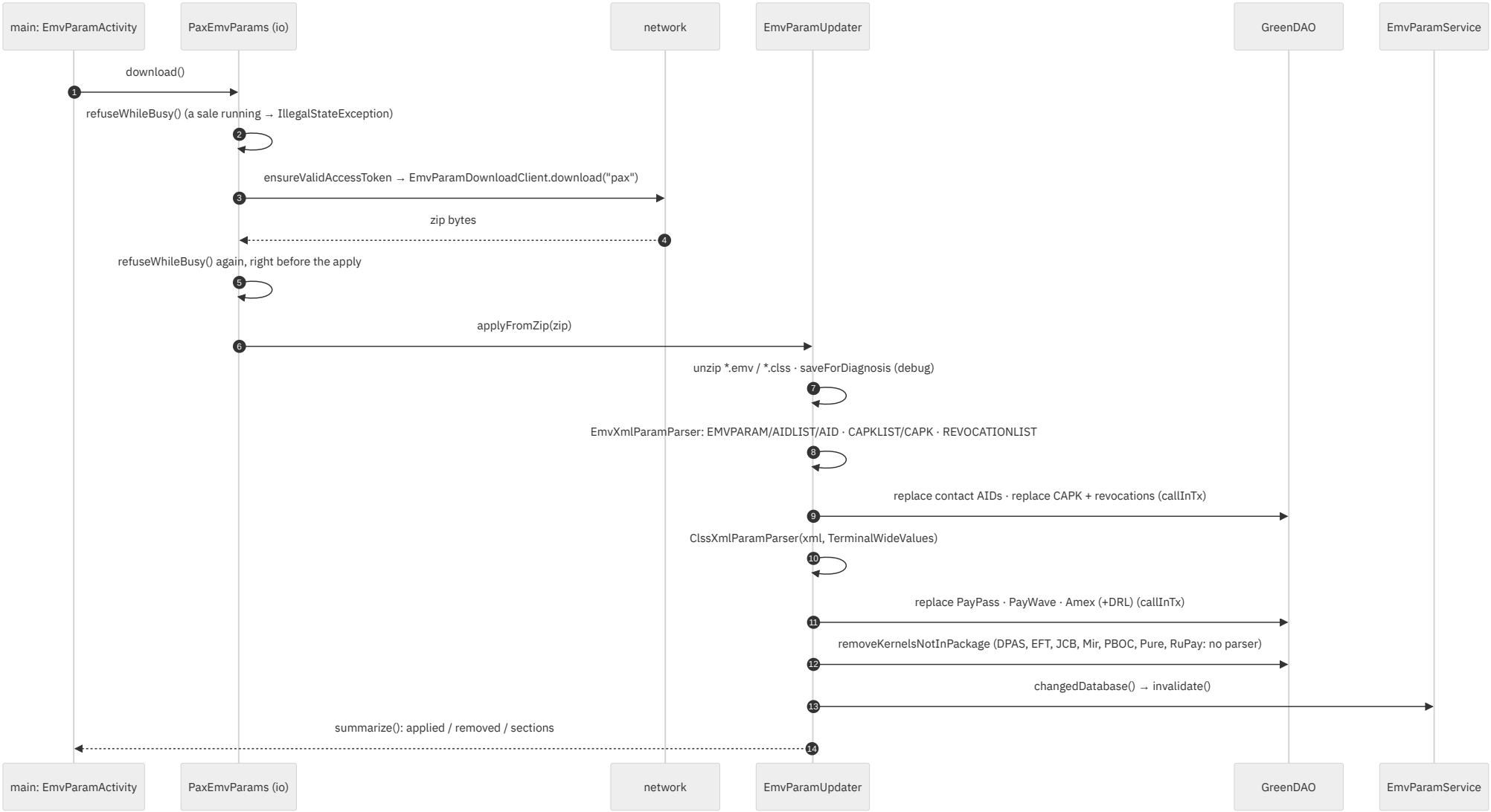
- **الـ contact:** إذا لم يتضمن ملف الـ host جزءًا معينًا، تبقى القيم القديمة كما هي. **الـ contactless:** ملف الـ host هو المرجع الكامل، وأي نوع بطاقة غير موجود فيه يُحذف.
- بيانات الجهاز (MCC، MID، TID، اسم التاجر) **لا** تأتي من ملف الـ host.
- ⚠ لا يوجد فحص لصلاحية الـ CAPKs: 76 من 88 في ملفات الـ JSON منتهية الصلاحية.

التفاصيل التقنية (للمراجعة)

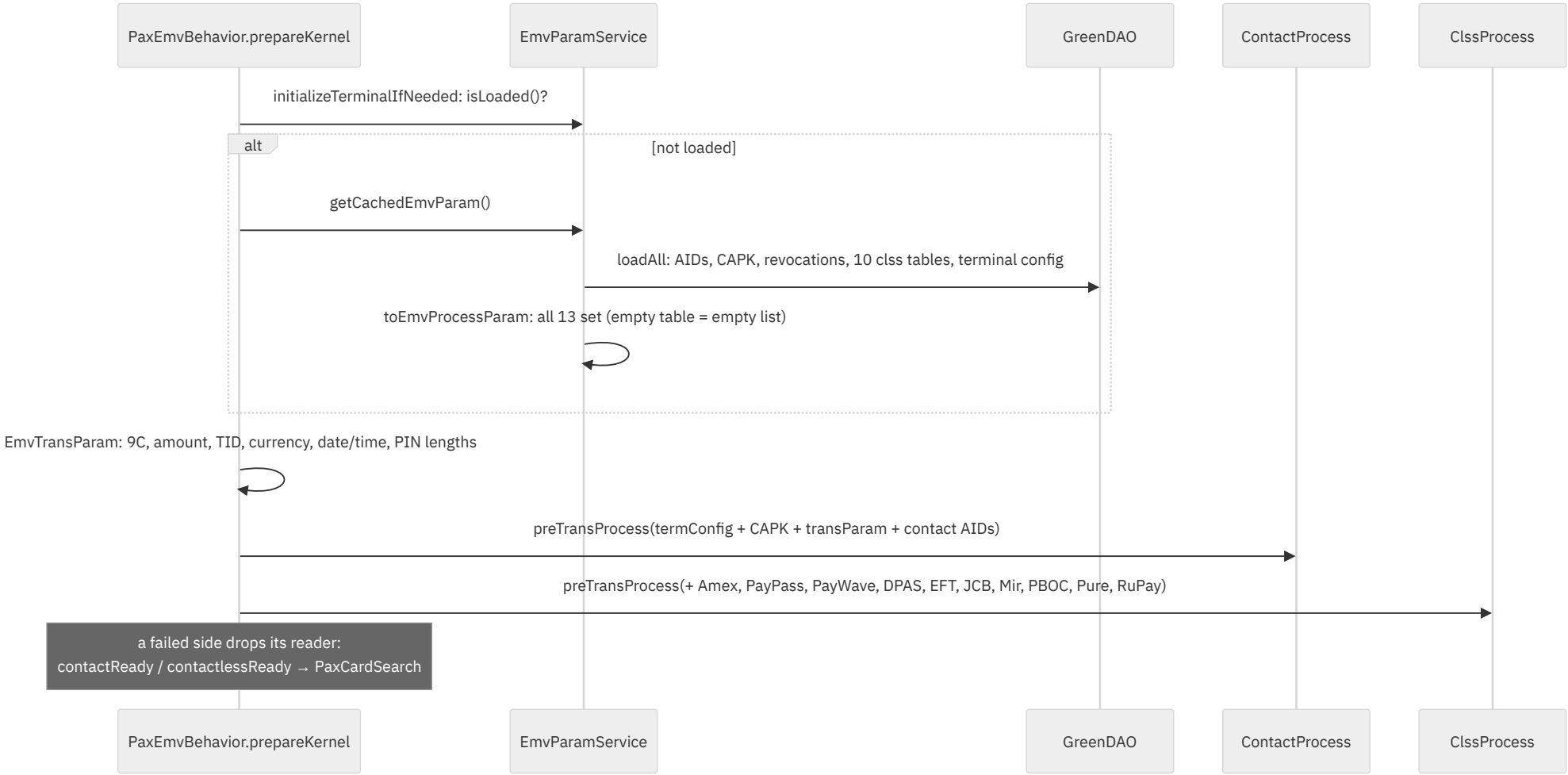
المسار بأسماء الكلاسات: يكتب `ConfigInit` (أول تشغيل) و `EmvParamUpdater.applyFromZip` (التنزيل) في GreenDAO؛ وتُفرغ `EmvParamService.invalidate()` الـ cache؛ → `getCachedEmvParam()` → `loadFromDatabase` → `toEmvProcessParam` (param 13)؛ `prepareKernel` → `preTransProcess` للـ contact والـ contactless.



التنزيل خطوة بخطوة


main: EmvParamActivity
PaxEmvParams (io)
network
EmvParamUpdater
GreenDAO
EmvParamService
download()
1
refuseWhileBusy() (a sale running → IllegalStateException)
2
ensureValidAccessToken → EmvParamDownloadClient.download("pax")
3
zip bytes
4
refuseWhileBusy() again, right before the apply
5
applyFromZip(zip)
6
unzip *.emv / *.clss · saveForDiagnosis (debug)
7
EmvXmlParamParser: EMVPARAM/AIDLIST/AID · CAPKLIST/CAPK · REVOCATIONLIST
8
replace contact AIDs · replace CAPK + revocations (callInTx)
9
ClssXmlParamParser(xml, TerminalWideValues)
10
replace PayPass · PayWave · Amex (+DRL) (callInTx)
11
removeKernelsNotInPackage (DPAS, EFT, JCB, Mir, PBOC, Pure, RuPay: no parser)
12
changedDatabase() → invalidate()
13
summarize(): applied / removed / sections
14
main: EmvParamActivity
PaxEmvParams (io)
network
EmvParamUpdater
GreenDAO
EmvParamService

أول عملية بعد التنزيل

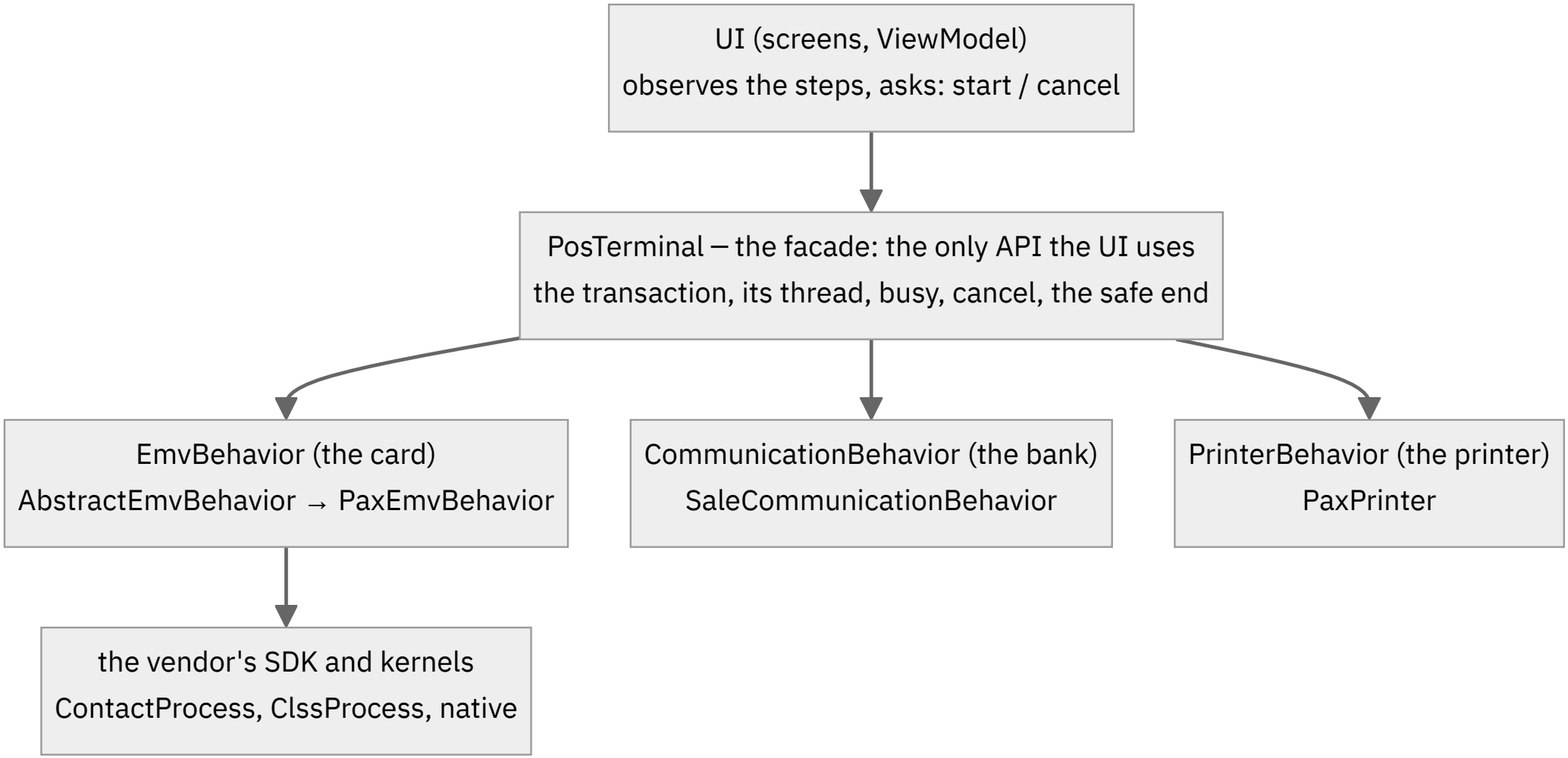

PaxEmvBehavior.prepareKernel
EmvParamService
GreenDAO
ContactProcess
ClssProcess
initializeTerminalIfNeeded: isLoaded()?
alt
[not loaded]
getCachedEmvParam()
loadAll: AIDs, CAPK, revocations, 10 clss tables, terminal config
toEmvProcessParam: all 13 set (empty table = empty list)
EmvTransParam: 9C, amount, TID, currency, date/time, PIN lengths
preTransProcess(termConfig + CAPK + transParam + contact AIDs)
preTransProcess(+ Amex, PayPass, PayWave, DPAS, EFT, JCB, Mir, PBOC, Pure, RuPay)
a failed side drops its reader:
contactReady / contactlessReady → PaxCardSearch
PaxEmvBehavior.prepareKernel
EmvParamService
GreenDAO
ContactProcess
ClssProcess

الـ param	أين يدخل الـ kernel
الـ contact AIDs، وأول AID كقيمة افتراضية	`ContactProcess.preTransProcess` (`EMVAddApp`، `setEmvAndMCKPara`)
9F33 / 9F40 / 9F35 الخاصة بالـ AID المختار	`emvSetParam` → `resetParam` (داخل `EMVReadAppData`)
CAPK و revocation	contact: `cardAuthentication` → `addCapk`؛ contactless: مرحلة الـ ODA
الـ contactless AIDs والحدود	`ClssEntryAddAid.addApp` + `Clss_PreTransProc_Entry`
TTQ، 9F33، floor، CVM limit، DF81xx	`setTransactionData` لكل (granular) kernel أو داخل الاستدعاء الواحد (atomic)
الـ TID في field 55 للـ contactless	`onReadCardOk` → `setTlv(9F1C)`

- `OTHERCONFIGURATION` (الـ TID و الـ MCC وبيانات التاجر) لا يُؤخذ من الـ XML، وهذا قرار مقصود.
- يُتجاهل أي CAPK بلا RID أو KeyID صحيح. ⚠ لا يوجد فحص للصلاحية ولا للـ checksum.
- أعاد `5755adb` الدوال الـ 12 `setXxxParam` الناقصة في `loadFromDatabase`، وهو إصلاح الـ crash في `preTransProcess`، ويحميه `EmvParamServiceCacheTest`.

10. مبادئ البرمجة في المشروع (ببساطة)

كل مبدأ في سطر: معناه ببساطة، وأين يظهر في الكود. لا شيء هنا للزينة؛ كل واحد يحل مشكلة حقيقية في الدفع.


UI (screens, ViewModel)
observes the steps, asks: start / cancel
PosTerminal — the facade: the only API the UI uses
the transaction, its thread, busy, cancel, the safe end
EmvBehavior (the card)
AbstractEmvBehavior → PaxEmvBehavior
CommunicationBehavior (the bank)
SaleCommunicationBehavior
PrinterBehavior (the printer)
PaxPrinter
the vendor's SDK and kernels
ContactProcess, ClssProcess, native

المبدأ / النمط	ببساطة	أين في الكود
Layers + Facade	طبقات، كل طبقة تكلم التي تحتها فقط. والـ UI يرى بابًا واحدًا يخفي ما خلفه	`PosTerminal` هو الباب الوحيد للـ UI (`startTran` / `steps()` / `acceptCard` / `saction()` / `cancel`). ليس طبقة تمرير فارغة: يملك العملية فعلًا (القاعدة 5 تمنع الطبقات التي تمرّر فقط)
Strategy / Polymorphism	واجهة واحدة وأكثر من تنفيذ، يُختار واحد منها	`EmvBehavior` (PAX / Ingenico / Fake)، `Commu`nicationBehavior`، PrinterBehavior`
Open / Closed	نضيف بدل أن نعدّل	`TerminalFactory` جديد بـ vendor جديد = flavor جديد وكلاساته، والـ core لا يتغيّر
Template Method	الكلاس الأب يحدد الترتيب، والابن يملأ الخطوات	`AbstractEmvBehavior` : `run` و `goToStep` والـ dispatch، و 16 `onXxx` يملؤها الـ vendor؛ `askThe`; `final Bank()`
Observer	من يهمه الأمر يشترك، ويصله كل جديد	`Transaction.steps()` (`ReplaySubject`) والشاشة مشتركة؛ `EmvEngine.emvSteps` و `EmvSt`epObserver`؛ `TransactionStepObserver`
Callback	تعطي جهة "رقمًا تتصل بك عليه"	`BankRequest` (§5)، `CardSearchListener` ، `I`ContactCallback` و `askStep` (§4)
Adapter	محوّل بين شكلين لا يعرف أحدهما الآخر	`PaxHostAnswer` (`AuthResult` → شكل PAX)؛ `ContactProcess.EmvCallBackListener` (native → listener)؛ `PaxEmvBehavior` بصفته `I`ContactCallback`
Factory / Builder	مكان واحد يقرر كيف يُنشأ الشيء	`TerminalFactory` (لكل flavor)، `ClssKernelPr`ocessFactory` (لكل kernel scheme)، `AuthResu`lt.approved / declined / failed / notSen`t`، `TransactionStepEvent.builder(...)`
Dependency Injection (يدوي)	الكلاس يأخذ ما يحتاجه من الخارج ولا يُنشئه	`PaxTerminal` → `super(behavior, communic`ation, printer)`؛ و `attach(this::sendToBa`nk)` للدائرة الوحيدة
Dependency Inversion (الـ D في SOLID)	الطبقة العليا تعتمد على واجهة، لا على كلاس محدد	`AbstractEmvBehavior` يعرف `BankRequest` ، لا `PosTerminal` ؛ الـ core يعرف `CommunicationBeh`avior`، لا Retrofit
Single Responsibility	كل كلاس له شغلة واحدة	`PaxCardSearch` (البحث)، `PaxPinEntry` (الـ PIN)، `PaxHostAnswer` ، `PaxReaders` ، `PaxCar`dData`، `PaxSetup` ، `PaxKernelSession`
Single source of truth	كل حالة لها مالك واحد ونسخة واحدة	الجهاز مشغول = `PosTerminal.current` ؛ حالة العملية كلها = `Transaction.end` ؛ خطواتها = `Tran`saction.steps`
Immutability / Value Object	الشيء لا يتغيّر بعد إنشائه؛ التعديل يعطي نسخة جديدة	`AuthResult` ، `TransactionStepEvent` ، `Tran`sactionConfig.withXxx(...)` يرجع config جديدًا
Fail closed	عند الشك نرفض، لا نوافق	لا رد = online failed؛ لا PIN = `USER_CANCEL` ؛ الـ callback يرد دائمًا (`setCallBackResult`)

11. استخدامنا كـ SDK (أي تطبيق يعمل integration)

الفكرة: أي تطبيق يستخدمنا للجزء بتاع الكارت والـ EMV. **أبسط عقد:** التطبيق يدّينا البنك بتاعه، ويسمع للـ steps. الـ `dispatchTransactionStep` (التأكيد والإلغاء) يفضل جوّانا، والتطبيق لا يكتبه.

الاتجاه	ماذا	كيف
التطبيق → إحنا	**البنك**	يكتب `CommunicationBehavior` : `connectToHo` إجباري (يبعت البيع ويرجّع `st(request)` `AuthResul`t)، و `acknowledge` / `reverse` اختياريين لو البنك بيدعمهم. الطلب يوصله وعليه الـ id (`request.g` `etAsyncRequestId()`) جاهز.
التطبيق → إحنا	الطباعة (اختياري)	`PrinterBehavior.print(lines)`
إحنا → التطبيق	**الـ steps**	`transaction.steps()` : يعمل subscribe ويعرض: `APPROVED` / `DECLINED` / `FAILED`، والنهاية `COMPLETED` / `ERROR` / `CANCELLED` (الجهاز متاح)، والباقي تقدّم.
التطبيق → إحنا	البدء والإلغاء	`pos.acceptCard(type, amount)` و `transact` `ion.cancel()`

```
// the integrator's bank — the only code they must write for the EMV part
class MyBank implements CommunicationBehavior {
    // required: send the sale, return the answer
    public Single<AuthResult> connectToHost(TransactionConfig request) { … }
    // optional
    public Completable acknowledge(TransactionConfig sent) { … }
    public Completable reverse(TransactionConfig sent) { … }
}

// their own screen (level B): start, observe, cancel
Transaction t = pos.acceptCard(TransactionType.SALE, amountMinor);
t.steps().subscribe(e → {
    switch (e.getStep()) {
        case APPROVED:  /* approved + receipt fields */ break;
        case DECLINED:  /* declined */ break;
        case FAILED:    /* the error */ break;
        case COMPLETED:
        case ERROR:
        case CANCELLED: /* done, terminal free */ break;
        default:        /* progress: CARD_READ, ONLINE_* … */ break;
    }
});
cancelButton.setOnClickListener(v → t.cancel());
```

- **ليه الـ switch بتاعنا يفضل جوّانا:** هو اللي بيحمي الفلوس. تأكيد لما الكارت يوافق، وإلغاء لما يرفض، وإلغاء في `ENDING` بعد cancel أو failure (الـ cancel ممكن ييجي والبنك لسه بيرد). لو كل تطبيق كتبه، واحد هينسى الإلغاء

بعد الـ cancel، فالفلوس تتخصم من العميل. جوّانا هو مكتوب مرة واحدة ومتغطّي بالـ tests.

- **الـ id بتاع التطبيق مش بتاعنا:** `sendToBank` بيسمّي الطلب (`asyncRequestId`) قبل ما يوصل للتطبيق، والتأكيد والإلغاء بيرجعوا بنفس الـ id. الـ EMV عمره ما بيشوفه.
- الـ switch اللي فوق في كود التطبيق للشاشة بتاعته بس. هو مش اللي بيحرّك العملية.

أبسط integration: بشاشة الـ search card بتاعتنا (level A)

التطبيق ما يكتبش شاشة خالص: يفتح شاشتنا بالمبلغ، ويستلم النتيجة.

```
// once, in their Application
class App extends Application implements TerminalProvider {
    private PosTerminal pos;

    public void onCreate() {
        super.onCreate();
        pos = TerminalFactory.create(this, new MyBank(), new MyPrinter());
    }

    public PosTerminal terminal() {
        return pos;
    }
}

// to take a payment: one launch, one result
ActivityResultLauncher<PaymentRequest> pay =
        registerForActivityResult(new EasyLinkPayment(), outcome → {
            // APPROVED / DECLINED / FAILED / CANCELLED,
            // the message, the masked PAN
        });
pay.launch(new PaymentRequest(TransactionType.SALE, amountMinor));
```

أسهل الخطوات عشان نطلّع الـ SDK والشاشة

#	الخطوة	ليه
1	`TerminalFactory.create(app, communicatio` `n, printer)`: البنك والطابعة يدخلوا من برّه. لو التطبيق ما بعتش حاجة، يفضل الـ default (`SaleComm` `unicationBehavior` و `PaxPrinter`).	النهارده `PaxTerminal()` بيعمل البنك والطابعة بنفسه، فالـ integrator مش هيقدر يدّينا البنك بتاعه. الـ constructor اللي بياخدهم موجود، بس `private` .
2	Android library module جديد (مثلًا `easylink-u` `i`) معتمد على `core` بس، فيه `SearchCardFragm` `ent` و `MainViewModel` (+ الـ Factory) و `Even` `t`، والـ layouts التلاتة (`fragment_search_card` و `item_entry_method` و `item_kv_row`) والـ strings بتاعتهم.	الشاشة النهارده جوّه `app` . نسمّي الـ resources بـ prefix (`easylink_`) عشان ما تتخانقش مع resources التطبيق.
3	interface اسمه `TerminalProvider` فيه `PosTer` `minal terminal()`. الـ `Application` بتاع التطبيق يعمله implement، و `EmvEnhanceApp` كمان.	ده الرابط الوحيد بين الشاشة والتطبيق. النهارده الشاشة بتعمل `((EmvEnhanceApp) getApplication()).` `getTerminal()`. وده interface ليه تطبيقين، فقاعدة 5 مش بتمنعه.
4	`PaymentActivity` (أو contract زي `EasyLinkPa` `yment`) شايل الـ fragment ويرجّع النتيجة بالـ Activity Result API.	النهارده زرار Done بيعمل `finish()` بس، والنتيجة ما بترجعش لحد.
5	الحاجات بتاعة الـ debug (الـ APDU trace على الشاشة وطباعته، ونافذة اختيار التطبيق حتى لو AID واحد) تشتغل في debug بس.	دي أدوات لينا، مش للـ integrator.
6	⚠ قبل ما نسلّم الـ SDK لأي حد: بوابة الـ release (BACKLOG §5): الـ PAN كامل و 55 field على الشاشة والإيصال، و logging `Level.BODY` ، و mToken حقيقي في الكود.	دي لو راحت لحد تاني، تبقى بيانات كروت متسربة.

الـ vendor (PAX) لسه بيتختار بالـ flavor و `TerminalFactory` ، فالـ integrator بياخد `core` و `vendor-` `pax` (ومكتباته) و `easylink-ui` . الكلام ده لسه اتجاه، ومفيش كود اتعمل له.

ملحوظة: الفرق مع `paxEmvBehaviorSplit`

الفرع الأقدم `paxEmvBehaviorSplit` (ومعه `-tests`) يعمل بالطريقة نفسها مع أسماء وتفاصيل أقدم: `authorizeOnline` / `Authorizer` / `PosTerminal.authorize` / `CommunicationBehavior.authorize` ؛ و `START_ONLINE_PROCESS` ؛ و `tx.lastAuth` في كل vendor؛ و `CLOSED` بدل `CLOSING_COMPLETED` ؛ ولا `ENDING` (النهاية عبر `onEnd` وحلقة)؛ والـ switch تستدعيه أكثر من جهة؛ والـ network بسلسلة Rx (`flatMap`) بلا `callTimeout` . القائمة الكاملة في `doc/REFERENCE.md` .